МИНОБРНАУКИ РОССИИ

РГУ НЕФТИ И ГАЗА (НИУ) ИМЕНИ И.М. ГУБКИНА

Факультет **Комплексной безопасности ТЭК**
Кафедра **Безопасности информационных технологий**

Оценка комиссии: _____ Рейтинг: _____
Подписи членов комиссии:

_____ _____
(подпись) (фамилия, имя, отчество)

_____ _____
(подпись) (фамилия, имя, отчество)

_____
(дата)

КУРСОВАЯ РАБОТА

по дисциплине Языки, методы и технологии программирования

на тему Разработка паттерна для обеспечения безопасности приложения при угрозе хищения обучающих данных

«К ЗАЩИТЕ»

Профессор, д.т.н. Корнеев Н.В.
(должность, ученая степень; фамилия, и.о.)

_____
(подпись)

_____
(дата)

ВЫПОЛНИЛ:
Студент группы **КС-23-18**
(номер группы)

Долгова Любовь Владимировна
(фамилия, имя, отчество)

_____
(подпись)

_____
(дата)

Москва, 20 25

МИНОБРНАУКИ РОССИИ
РГУ НЕФТИ И ГАЗА (НИУ) ИМЕНИ И.М. ГУБКИНА

Факультет **Комплексной безопасности ТЭК**
Кафедра **Безопасности информационных технологий**

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

по дисциплине Языки, методы и технологии программирования

на тему Разработка паттерна для обеспечения безопасности приложения при угрозе хищения обучающих данных

ДАНО студенту **Долговой Любови Владимировне** (фамилия, имя, отчество в дательном падеже) группы **КС-23-18** (номер группы)

Содержание работы:
1. Введение
2. Анализ предметной области
3. Разработка паттерна
4. Разработка микросервисов
5. Тестирование и мониторинг микросервисов на кластере
6. Заключение
7. Приложение

Исходные данные для выполнения работы:
1. Информация о предметной области из открытых источников

Рекомендуемая литература:
1. Положение о курсовом проектировании в РГУ нефти и газа (НИУ) имени И.М. Губкина (приказ от 11.11.2021 № 398)
2. Docker Compose Documentation [Электронный ресурс]: сайт. – Режим доступа: https://docs.docker.com/compose/
3. Kubernetes Documentation [Электронный ресурс]: сайт. – Режим доступа: https://kubernetes.io/docs/home/

Графическая часть:
1. Модель угрозы; 2. Механизм защиты; 3. Архитектура паттерна; 4. UML диаграммы; 5. Коды микросервисов; 6. Витрины с результатами мониторинга и тестирования микросервисов на кластере

Требования к представлению результатов:
Электронная версия

Руководитель: д.т.н. (уч.степень) профессор (должность) _____ (подпись) Корнеев Николай Владимирович (фамилия, имя, отчество)

Задание принял к исполнению: студент _____ (подпись) Долгова Любовь Владимировна (фамилия, имя, отчество)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**
ГЛАВА 1. Анализ предметной области **9**
1.1 Модели угроз хищения обучающих данных **9**
1.2 Механизмы защиты сценариев хищения обучающих данных **13**
ГЛАВА 2. РАЗРАБОТКА ПАТТЕРНА **16**
2.1 Архитектура паттерна обеспечения безопасности приложения **16**
2.2 UML-диаграммы **19**
ГЛАВА 3. РАЗРАБОТКА МИКРОСЕРВИСОВ **23**
3.1 Функции микросервисов **23**
3.2 Развертывание микросервиса **44**
4. ТЕСТИРОВАНИЕ И МОНИТОРИНГ МИКРОСЕРВИСОВ НА КЛАСТЕРЕ **52**
4.1 Тестирование микросервисов **52**
4.2 Мониторинг микросервисов **57**
ЗАКЛЮЧЕНИЕ **63**
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **65**
ПРИЛОЖЕНИЕ А **71**

ВВЕДЕНИЕ

В условиях стремительного развития цифровых технологий и активного внедрения методов машинного обучения в различные сферы деятельности возрастает значение вопросов обеспечения информационной безопасности интеллектуальных приложений приобретают первостепенное значение. Современные системы машинного обучения широко применяются в банковском секторе, промышленности, медицине, государственном управлении и других областях, где обрабатываются большие объёмы данных. Одним из ключевых компонентов таких систем являются обучающие данные, используемые для построения и адаптации моделей машинного обучения. Именно обучающие данные во многом определяют точность, устойчивость и корректность работы интеллектуальных алгоритмов.

Данная работа посвящена разработке проектного паттерна, направленного на обеспечение безопасности приложений, использующих технологии машинного обучения, в условиях угрозы хищения обучающих данных. В рамках исследования проведен всесторонний анализ существующих подходов и методов защиты информации, а также выявлены ключевые уязвимости, возникающие при хранении, передачи и использовании обучающих данных в микросервисных архитектурах.

На основе полученных данных разработан архитектурный паттерн, ориентированный на защиту обучающих данных на уровне проектирования приложения и включающий механизмы контроля доступа, мониторинга и аудита, мониторинга и аудита операций с данными, а также средства обеспечения конфиденциальности и целостности обучающих данных.

Предложенный паттерн представляет собой совокупность архитектурных и программных решений, обеспечивающих многоуровневую защиту обучающих данных на различных этапах их жизненного цикла. Внедрение данного паттерна позволяет значительно снизить вероятность несанкционированного доступа к обучающим данным и минимизировать потенциальные последствия их утечки. Результаты исследования могут быть использованы при разработке и модернизации приложений машинного

обучения, функционирующих в условиях повышенных требований к информационной безопасности.

Актуальность темы обусловлена тем, что обучающие данные представляют собой ценный и уязвимый ресурс, хищение которого может привести к серьёзным негативным последствиям. В отличие от традиционных пользовательских данных, утечка обучающих датасетов способна не только нарушить конфиденциальность информации, но и привести к утрате уникальных наборов данных, формировавшихся в течение длительного времени. Кроме того, компрометация обучающих данных может позволить злоумышленникам воспроизвести модели машинного обучения, провести анализ используемых признаков или использовать полученные данные для создания конкурентных или вредоносных решений.

Дополнительную сложность обеспечивает архитектура современных приложений машинного обучения, которые часто реализуются в виде распределённых микросервисных систем и развертываются в облачных или гибридных инфраструктурах. Обучающие данные в таких системах передаются между различными сервисами, хранятся в централизованных хранилищах и используются в автоматизированных конвейерах обучения и обновления моделей. Это формирует дополнительные векторы атак и повышает риск несанкционированного доступа к данным на различных этапах их жизненного цикла.

Существующие подходы к обеспечению безопасности информационных систем в основном ориентированы на защиту конечных пользовательских данных и традиционных баз данных, что не всегда учитывает специфику обработки обучающих данных в системах машинного обучения. В связи с этим возникает необходимость разработки специализированных архитектурных решений, позволяющих обеспечить комплексную защиту обучающих данных на уровне проектирования приложения. Разработка паттерна обеспечения безопасности приложения при угрозе хищения обучающих данных является актуальной задачей, направленной на повышение устойчивости современных интеллектуальных информационных систем к киберугрозам.

Целью данной работы является разработка архитектурного паттерна обеспечения безопасности приложения машинного обучения при угрозе хищения обучающих данных.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ специфики угроз хищения обучающих данных как объекта защиты в приложениях машинного обучения;
2. Исследовать актуальные угрозы хищения обучающих данных и способы их реализации с использованием методических указаний ФСТЭК России;
3. Построить модель угроз безопасности информации для приложения, использующего обучающие данные;
4. Разработать архитектурный паттерн защиты на основе микросервисного подхода, направленный на предотвращение хищения обучающих данных;
5. Реализовать программный прототип разработанного паттерна с использованием контейнеризации и оркестрации;
6. Провести тестирование и мониторинг микросервисов с использованием специализированных метрик безопасности.

Объектом исследования являются приложения и информационные системы, использующие технологии машинного обучения и обрабатывающие обучающие данные.

Предметом исследования выступают архитектурные и программные методы обеспечения безопасности обучающих данных в приложениях машинного обучения при угрозе их хищения.

Новизна работы заключается в разработке специализированного паттерна безопасности, ориентированного на защиту обучающих данных на уровне архитектуры приложения. В отличие от универсальных подходов к защите данных, предлагаемый паттерн учитывает особенности жизненного цикла обучающих данных, их использование в распределенных микросервисных архитектурах и необходимость централизованного контроля доступа, мониторинга и аудита операций с датасетами.

Практическая ценность данной курсовой работы заключается в разработке архитектурного паттерна обеспечения безопасности приложения при угрозе хищения обучающих данных, используемые в системах машинного обучения. В современных информационных системах обучающие данные являются критически важным ресурсом, поскольку определяют качество и корректность функционирования моделей машинного обучения, а также могут содержать конфиденциальную или коммерчески значимую информацию. В связи с этим защита обучающих данных на этапе проектирования приложения приобретает особую практическую значимость.

Разработанный в работе паттерн может быть использован при проектировании и внедрении приложений машинного обучения, развертываемых в облачных и серверных инфраструктурах. Предложенная микросервисная архитектура с выделенным сервисом безопасности позволяет централизовать контроль доступа к обучающим данным, обеспечить их защиту на этапах хранения, передачи и использования, а также реализовать механизмы аудита и мониторинга операций с датасетами. Это снижает вероятность несанкционированного доступа и утечки обучающих данных в распределённых системах.

Практическая значимость работы также заключается в возможности применения предложенного паттерна в процессах разработки и эксплуатации ML-приложений, включая конвейеры MLOps. Реализованный программный прототип демонстрирует возможность интеграции механизмов безопасности в контейнеризированную среду с использованием Docker и Kubernetes, что делает решение применимым в реальных условиях эксплуатации.

Кроме того, материалы работы могут быть использованы в учебных целях при изучении архитектурных паттернов проектирования, микросервисного подхода и вопросов обеспечения безопасности информационных систем и машинного обучения. Разработанный паттерн может служить практическим примером применения методов моделирования угроз и архитектурных решений для защиты обучающих данных в современных приложениях.

Структурно курсовая работа состоит из введения, трех глав и заключения. В первой главе проводится анализ предметной области и моделирование угроз безопасности обучающих данных. Во второй главе разрабатывается архитектурный паттерн обеспечения безопасности приложения. В третьей главе рассматривается реализация микросервисов, их развертывание, тестирование и мониторинг. В заключении подводятся итоги работы и формулируются выводы по достижению поставленных целей и задач.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 МОДЕЛИ УГРОЗ ХИЩЕНИЯ ОБУЧАЮЩИХ ДАННЫХ

В современных приложениях машинного обучения обучающие данные являются одним из ключевых компонентов, определяющих качество, точность и устойчивость моделей. Под обучающими данными понимаются наборы структурированных и неструктурированных данных, используемые для обучения, дообучения и валидации моделей машинного обучения. Такие данные могут включать результаты измерений, статистические показатели, текстовую и графическую информацию, а также данные, полученные в результате экспертной разметки.

Особенностью обучающих данных является их высокая ценность как объекта интеллектуальной собственности. Формирование обучающих датасетов зачастую требует значительных временных, финансовых и человеческих ресурсов. В ряде случаев обучающие данные могут содержать персональные данные или коммерчески чувствительную информацию, что накладывает дополнительные требования к их защите в соответствии с нормативными документами в области информационной безопасности.

В отличие от традиционных пользовательских данных, обучающие данные активно используются в автоматизированных процессах, передаются между компонентами приложения и могут многократно копироваться в процессе обучения и тестирования моделей. Это приводит к расширению поверхности атаки и увеличивает риск их компрометации. В распределённых микросервисных архитектурах обучающие данные хранятся в централизованных хранилищах и используются различными сервисами, что требует комплексного подхода к обеспечению их безопасности.

В рамках данной работы моделирование угроз хищения обучающих данных осуществляется в соответствии с методикой ФСТЭК России, изложенной в документе «Методика оценки угроз безопасности информации». Согласно данной методике угрозы рассматриваются с точки зрения нарушения основных свойств информации: конфиденциальности, целостности и доступности. Для обучающих данных наибольшую актуальность представляют

угрозы нарушения конфиденциальности и целостности, поскольку их реализация может привести к утечке данных и компрометации моделей машинного обучения.

На основе изложенных положений в рамках данной работы рассмотрены два сценария реализации угроз хищения обучающих данных, соответствующие требованиям методики ФСТЭК России.

Сценарий 1: Реализация угрозы хищения обучающих данных вследствие нарушения механизмов контроля доступа

Первый сценарий угрозы рассматривает возможность хищения обучающих данных в результате несанкционированного доступа к хранилищу обучающих датасетов через уязвимый сервис приложения машинного обучения. Данный сценарий относится к угрозам нарушения конфиденциальности информации и может быть реализован как внешним, так и внутренним нарушителем.

В типовой архитектуре ML-приложения доступ к обучающим данным осуществляется через специализированные сервисы обработки и подготовки данных. При отсутствии должного контроля доступа или при наличии уязвимостей в интерфейсах взаимодействия злоумышленник может получить доступ к функционалу, позволяющему обращаться к хранилищу обучающих данных.

На рисунке 1 представлена схема сценария хищения обучающих данных через несанкционированный доступ к хранилищу.



Рисунок 1 – Сценарий хищения обучающих данных через несанкционированный доступ к хранилищу.

Описание этапов сценария:

1. **Инициация атаки:** Злоумышленник осуществляет анализ приложения машинного обучения и выявляет сервисы, взаимодействующие с обучающими данными, включая API обработки или подготовки данных.
2. **Эксплуатация уязвимости:** В результате некорректной настройки механизмов аутентификации или авторизации злоумышленник получает возможность обращения к сервису без соответствующих прав доступа.
3. **Доступ к хранилищу обучающих данных:** Используя уязвимый сервис, нарушитель инициирует запросы к хранилищу обучающих данных, минуя предусмотренные механизмы контроля доступа.
4. **Хищение обучающих данных:** Злоумышленник осуществляет копирование обучающих датасетов или части на внешний ресурс, что приводит к нарушение конфиденциальности информации.
5. **Последствия реализации угрозы:** В результате утечки обучающих данных возникает риск компрометации моделей машинного обучения, утраты интеллектуальной собственности и нарушения требований информационной безопасности.

Сценарий 2: Реализация угрозы хищения обучающих данных при компрометации учетных данных привилегированного пользователя

Второй сценарий угрозы связан с компрометацией учётной записи пользователя, обладающего расширенными правами доступа к обучающим данным, например ML-инженера или администратора ML-платформы. Данный сценарий относится к угрозам нарушения конфиденциальности и целостности информации и может быть реализован с использованием методов социальной инженерии или компрометации учётных данных.

В условиях распределённых систем машинного обучения пользователи с расширенными правами доступа имеют возможность выполнять операции по выгрузке, модификации и удалению обучающих данных. Компрометация их учётных данных позволяет злоумышленнику легитимно взаимодействовать с системой, что затрудняет обнаружение атаки.

На рисунке 2 представлена схема сценария хищения обучающих данных при компрометации учётной записи.

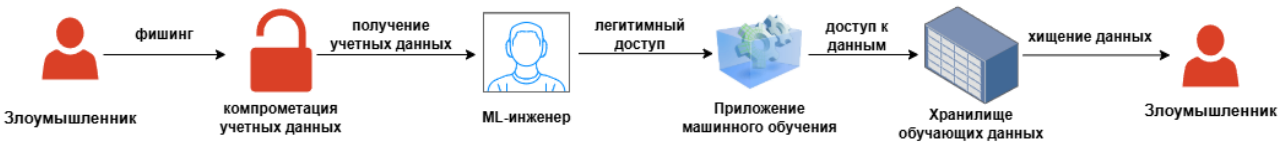


Рисунок 2 – Сценарий хищения обучающих данных при компрометации учетной записи пользователя.

Описание этапов сценария:

1. **Подготовка атаки:** Злоумышленник осуществляет фишинговую атаку либо иным способом получает учетные данные пользователя с расширенными правами доступа.
2. **Компрометация учетной записи:** Полученные учетные данные используются для легитимного входа в систему машинного обучения.
3. **Получения доступа к обучающим данным:** Злоумышленник получает доступ к функционалу управления обучающими данными, включая операции их выгрузки и изменения.
4. **Хищение или модификация данных:** Обучающие данные копируются на внешний ресурс либо подвергаются несанкционированному изменению, что приводит к нарушению конфиденциальности и целостности информации.
5. **Последствия реализации угрозы:** Компрометация обучающих данных может привести к искажению результатов обучения моделей, утрате доверия к системе и финансовым потерям.

Рассмотренные сценарии демонстрируют актуальные угрозы хищения обучающих данных в приложениях машинного обучения и подтверждают необходимость применения специализированных архитектурных решений для их защиты. Для нейтрализации выявленных угроз в следующем подразделе рассматривается механизм защиты, основанный на проектном паттерне обеспечения безопасности приложения.

1.2 МЕХАНИЗМЫ ЗАЩИТЫ СЦЕНАРИЕВ ХИЩЕНИЯ ОБУЧАЮЩИХ ДАННЫХ

Рассмотренные в пункте 1.1 сценарии реализации угроз хищения обучающих данных показывают, что основными причинами успешной реализации атак являются отсутствие централизованного контроля доступа к данным, возможность обхода механизмов проверки запросов, а также компрометация учётных данных пользователей с расширенными правами. В условиях микросервисной архитектуры приложения машинного обучения данные факторы приводят к увеличению риска несанкционированного доступа к обучающим данным и их утечки.

В соответствии с методикой ФСТЭК России, для противодействия выявленным угрозам необходимо реализовать механизм защиты, обеспечивающий контроль доступа к обучающим данным, мониторинг операций и ограничение последствий компрометации отдельных компонентов системы. В рамках данной работы предлагается механизм защиты, основанный на внедрении специализированного сервиса безопасности, через который осуществляется взаимодействие с хранилищем обучающих данных.

Механизм защиты при несанкционированном доступе к сервису обработке данных

Для нейтрализации сценария 1, связанного с попыткой хищения обучающих данных через уязвимый сервис обработки данных, предлагаемый механизм защиты предусматривает обязательное прохождение всех запросов через сервис безопасности. Данный сервис выполняет функции аутентификации и авторизации, а также проверку допустимости операций с обучающими данными.

В рамках данного механизма сервис обработки данных не имеет прямого доступа к хранилищу обучающих данных. Любой запрос на получение или обработку обучающих данных перенаправляется в сервис безопасности, где осуществляется проверка прав доступа и соответствия запроса установленным

политикам безопасности. Только после успешного прохождения всех проверок запрос передаётся в хранилище обучающих данных.

Дополнительно реализуется аудит операций доступа к данным, что позволяет фиксировать все попытки обращения к обучающим данным и выявлять подозрительные действия. Такой подход исключает возможность обхода механизмов защиты и существенно снижает риск реализации угрозы хищения обучающих данных при эксплуатации уязвимостей сервисов приложения.

Схема механизма защиты для данного сценария представлена на рисунке 3.



Рисунок 3 – Механизм защиты обучающих данных при несанкционированном доступе к сервису обработки данных.

Механизм защиты при компрометации учетных данных пользователя

Для противодействия сценарию 2, связанному с компрометацией учётных данных пользователя с расширенными правами доступа (ML-инженера или администратора ML-платформы), механизм защиты дополняется средствами мониторинга действий пользователя и контроля аномального поведения. Даже при успешной аутентификации пользователя доступ к обучающим данным осуществляется в рамках ограниченных политик безопасности.

В случае выполнения нетипичных операций, таких как массовая выгрузка обучающих данных или обращение к данным вне стандартного рабочего

контекста, сервис безопасности фиксирует инцидент и инициирует защитные меры. К таким мерам относятся временное ограничение доступа к обучающим данным, завершение активной сессии пользователя и регистрация события в системе аудита.

Таким образом, даже при компрометации учётных данных злоумышленник не получает возможности бесконтрольного доступа к обучающим данным, а последствия инцидента минимизируются за счёт оперативного реагирования системы безопасности.

Схема механизма защиты для данного сценария представлена на рисунке 4.

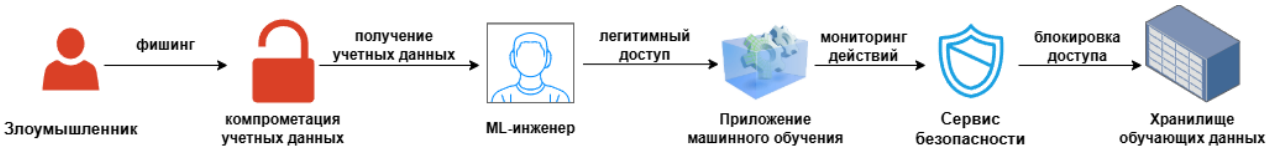


Рисунок 4 – Механизм защиты обучающих данных при компрометации учетных данных пользователя.

Рассмотренные механизмы защиты обеспечивают противодействие основным сценариям хищения обучающих данных, выявленным в пункте 1.1. Централизация функций безопасности, контроль операций доступа к данным и мониторинг действий пользователей позволяют повысить уровень защищённости приложений машинного обучения и снизить риск утечки обучающих данных.

Полученные выводы и сформулированные требования к механизму защиты используются в следующей главе при разработке архитектурного паттерна обеспечения безопасности приложения при угрозе хищения обучающих данных.

ГЛАВА 2. РАЗРАБОТКА ПАТТЕРНА

В первой главе курсовой работы были рассмотрены обучающие данные как объект защиты, а также проанализированы основные сценарии реализации угроз их хищения в приложениях машинного обучения. Проведенный анализ показал, что ключевыми факторами, способствующими утечке обучающих данных, являются отсутствие централизованного контроля доступа, возможность обхода механизмов безопасности и компрометация учетных данных пользователей с расширенными правами.

В связи с этим возникает необходимость разработки архитектурного паттерна, ориентированного на обеспечение безопасности приложения машинного обучения на уровне проектирования. Использование архитектурного паттерна позволяет формализовать подход к защите обучающих данных, обеспечить повторяемость проектных решений и снизить вероятность ошибок при разработке и масштабировании приложения.

В рамках данной главы разрабатывается архитектурный паттерн обеспечения безопасности приложения при угрозе хищения обучающих данных, основанный на микросервисной архитектуре и предусматривающий использование ограниченного набора специализированных микросервисов для реализации функций контроля доступа, мониторинга и защиты данных.

2.1 АРХИТЕКТУРА ПАТТЕРНА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИЛОЖЕНИЯ

Разрабатываемый архитектурный паттерн ориентирован на защиту обучающих данных как ключевого ресурса приложения машинного обучения и реализует принцип централизованного контроля доступа. В рамках паттерна используется микросервисная архитектура, позволяющая изолировать функции безопасности, обработки данных и мониторинга, а также обеспечить масштабируемость и устойчивость системы к компрометации отдельных компонентов.

Архитектура паттерна построена на основе пяти взаимосвязанных микросервисов, каждый из которых выполняет строго определенную роль в обеспечении безопасности обучающих данных. Использование фиксированного

количества микросервисов позволяет упростить управление системой, повысить прозрачность архитектуры и обеспечить соответствие требованиям задания. В дальнейшем данные микросервисы образуют единый защищённый контур, через который осуществляется взаимодействие с хранилищем обучающих данных.

Все микросервисы архитектурного паттерна предполагается развёртывать в контейнеризированной среде с использованием технологии Docker. Для управления жизненным циклом контейнеров и обеспечения согласованного взаимодействия микросервисов используется система оркестрации Kubernetes. Применение данных технологий позволяет реализовать изоляцию компонентов, масштабирование сервисов безопасности и устойчивость системы к отказам отдельных элементов.

На рисунке 5 представлена архитектура паттерна обеспечения безопасности приложения при угрозе хищения обучающих данных.

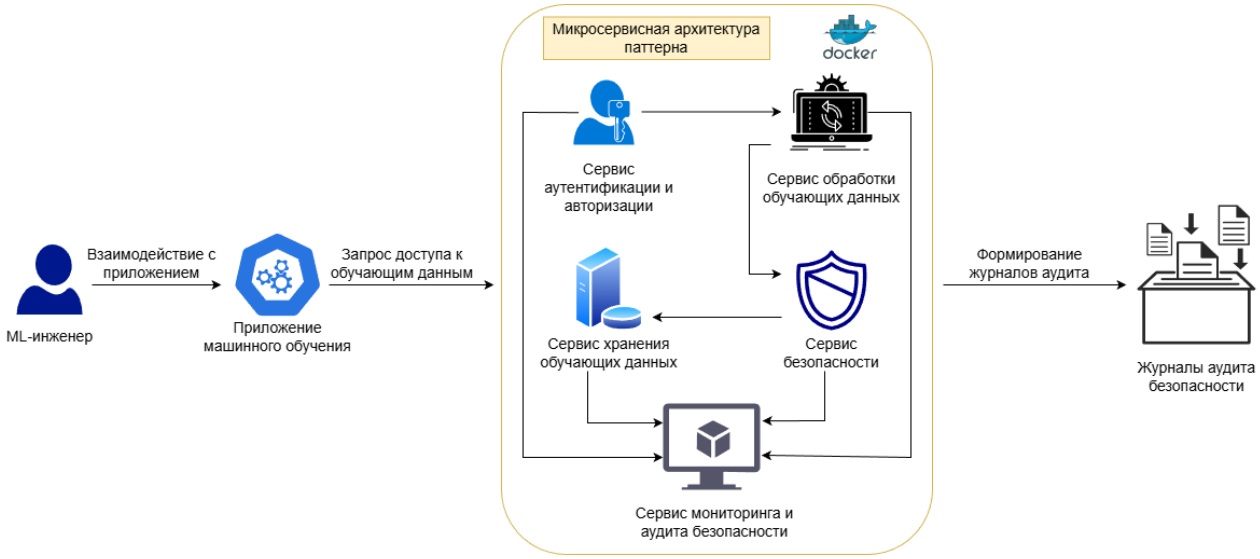


Рисунок 5 – Микросервисная архитектура паттерна безопасности

Взаимодействие пользователей с системой осуществляется через приложение машинного обучения, которое инициирует обращения к микросервисам безопасности и обработки данных.

Проверка подлинности и прав доступа пользователей выполняется сервисом аутентификации и авторизации, после чего запросы на обработку обучающих данных передаются в соответствующий сервис обработки. Для

получения доступа к обучающим данным сервис обработки обращается к сервису безопасности, который принимает решение о разрешении или блокировании доступа на основе заданных политик безопасности.

Доступ к сервису хранения обучающих данных осуществляется исключительно через сервис безопасности, что исключает возможность прямого обращения к хранилищу и снижает риск несанкционированного копирования или утечки данных. Все действия, связанные с доступом к обучающим данным, а также события, имеющие отношение к информационной безопасности, фиксируются сервисом мониторинга и аудита безопасности.

Результаты работы сервиса мониторинга выводятся за пределы микросервисной архитектуры в виде журналов аудита безопасности, предназначенных для последующего анализа, выявления инцидентов и расследования нарушений. Таким образом, предложенная архитектура паттерна обеспечивает централизованный контроль доступа, изоляцию критически важных компонентов и повышение устойчивости приложения машинного обучения к угрозам хищения обучающих данных.

Состав микросервисов архитектурного паттерна

1. Сервис аутентификации и авторизации

Данный микросервис отвечает за проверку подлинности пользователей и сервисов, а также за определение их прав доступа к ресурсам системы. Аутентификация осуществляется на основе учётных данных, а авторизация — в соответствии с заданными ролями и политиками безопасности. Сервис предотвращает несанкционированный доступ к функционалу приложения и является первой линией защиты.

2. Сервис обработки обучающих данных

Микросервис выполняет операции подготовки обучающих данных, включая фильтрацию, агрегацию и формирование выборок для обучения моделей машинного обучения. Данный сервис не имеет прямого доступа к хранилищу данных и взаимодействует с ним исключительно через сервис безопасности, что исключает возможность обхода механизмов защиты.

3. Сервис безопасности

Центральный элемент архитектурного паттерна, обеспечивающий контроль всех операций доступа к обучающим данным. Сервис принимает решения о разрешении или блокировании запросов на основе политик безопасности, контекста запроса и информации, полученной от других микросервисов. Именно данный компонент обеспечивает нейтрализацию сценариев угроз, рассмотренных в главе 1.

4. Сервис мониторинга и аудита безопасности

Предназначен для регистрации и анализа событий, связанных с доступом к обучающим данным. Сервис фиксирует все попытки доступа, выявляет аномалии в поведении пользователей и сервисов, а также формирует журналы аудита, используемые для расследования инцидентов и оценки уровня защищённости системы.

5. Сервис хранения обучающих данных

Представляет собой защищённое хранилище, предназначенное для размещения наборов обучающих данных. Доступ к данному сервису возможен только через сервис безопасности, что исключает прямое обращение со стороны других микросервисов и снижает риск утечки данных при компрометации отдельных компонентов системы.

Таким образом, предложенная архитектура паттерна обеспечивает чёткое разделение ответственности между микросервисами и создаёт единый контролируемый контур доступа к обучающим данным. Это позволяет повысить уровень безопасности приложения машинного обучения и обеспечить устойчивость системы к угрозам хищения данных.

2.2 UML-ДИАГРАММЫ

В рамках разработки архитектурного паттерна обеспечения безопасности приложения при угрозе хищения обучающих данных важным этапом является формализация взаимодействия компонентов системы. Для наглядного представления последовательности обработки запросов и принятия решений в условиях возможных угроз используется UML-диаграмма последовательности. Данный вид диаграмм позволяет отразить порядок обмена сообщениями между

участниками системы, а также определить точки принятия решений и фиксации событий безопасности.

На рисунке 6 представлена UML-диаграмма последовательности, иллюстрирующая процесс обработки запроса на доступ к обучающим данным в рамках разработанного паттерна безопасности. В качестве инициатора запроса выступает нарушитель, взаимодействующий с приложением машинного обучения. Далее запрос проходит последовательную проверку через сервисы,, где осуществляется анализ запроса и проверка соответствия политикам безопасности.

В диаграмме предусмотрено два альтернативных сценария обработки запроса: при обнаружении угрозы безопасности выполняется блокировка доступа с последующим уведомлением приложения и регистрацией события в журналах аудита безопасности; при отсутствии угрозы осуществляется разрешённый доступ к обучающим данным с обязательной фиксацией факта обращения в системе мониторинга и аудита. Таким образом, диаграмма демонстрирует реализацию централизованного контроля доступа и обязательного аудита всех операций, связанных с использованием обучающих данных.

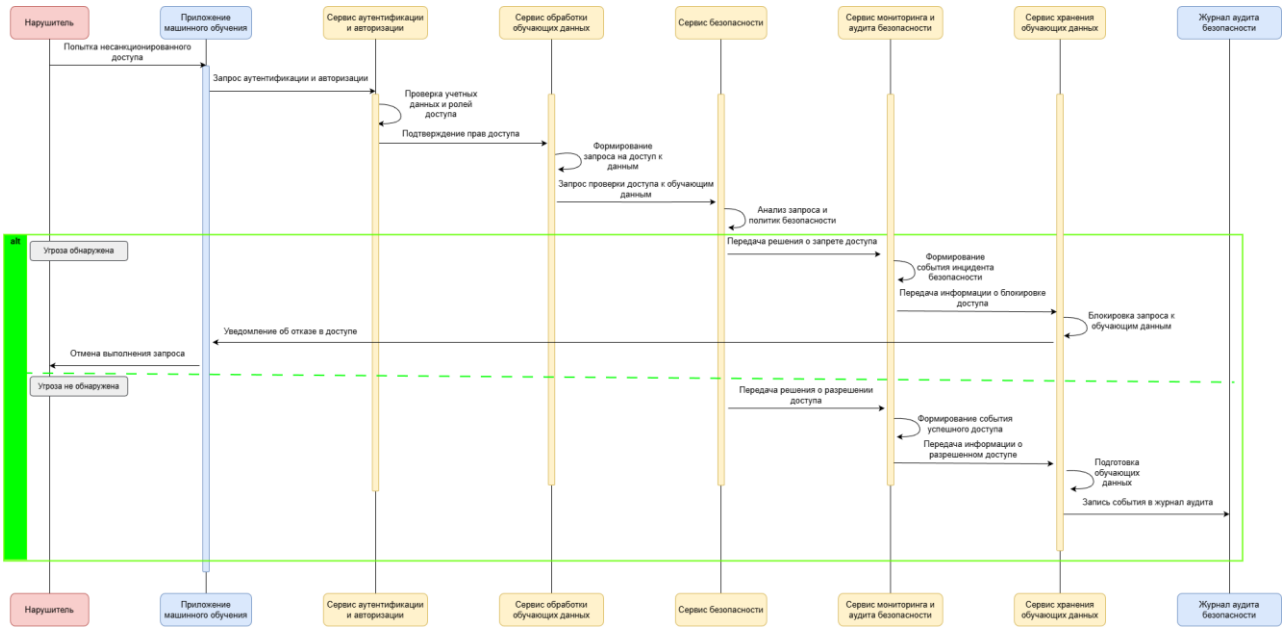


Рисунок 6 – UML-диаграмма последовательности паттерна безопасности

На рисунке 7 представлена UML-диаграмма состояний разработанного паттерна обеспечения безопасности приложения машинного обучения при угрозе хищения обучающих данных. Диаграмма отражает логику обработки запроса доступа к обучающим данным на уровне механизмов аутентификации, контроля доступа и анализа угроз безопасности.

Процесс обработки запроса начинается с проверки аутентификации, в ходе которой выполняется валидация токена доступа. При успешной аутентификации запрос передаётся на этап проверки политик доступа, где анализируется соответствие прав пользователя установленным ограничениям, а при невалидном токене или недостаточности прав инициируется блокировка доступа. Если политики доступа разрешают выполнение запроса, осуществляется анализ угроз безопасности с целью выявления признаков хищения обучающих данных: при обнаружении угрозы запрос блокируется и регистрируется как инцидент безопасности в журнале аудита, а при отсутствии угроз доступ к обучающим данным предоставляется и выполнение запроса завершается успешно.

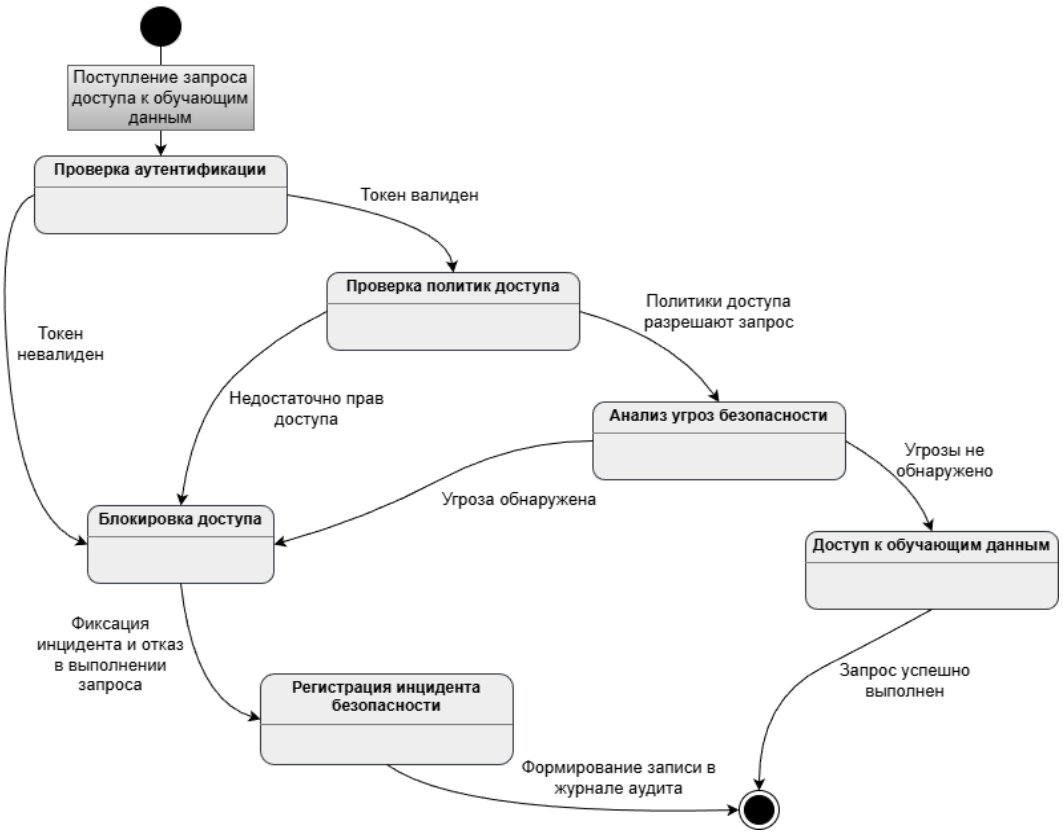


Рисунок 7 – UML-диаграмма состояния паттерна безопасности

На рисунке 8 представлена UML-диаграмма деятельности, которая отражает процесс функционирования архитектурного паттерна обеспечения безопасности приложения машинного обучения при угрозе хищения обучающих данных. Процесс начинается с формирования пользователем запроса на доступ к данным и его передачи в приложение машинного обучения, после чего выполняется аутентификация и авторизация.

В случае неуспешной аутентификации инициируется блокировка доступа и выполнение запроса завершается. При успешной проверке запрос передаётся на обработку и анализ в сервис безопасности, где осуществляется проверка политик доступа и выявление возможных угроз хищения данных. При обнаружении угрозы доступ блокируется и событие регистрируется в системе аудита. При отсутствии угроз доступ к обучающим данным разрешается, данные предоставляются, а факт доступа фиксируется в журнале аудита, после чего результат запроса возвращается пользователю.

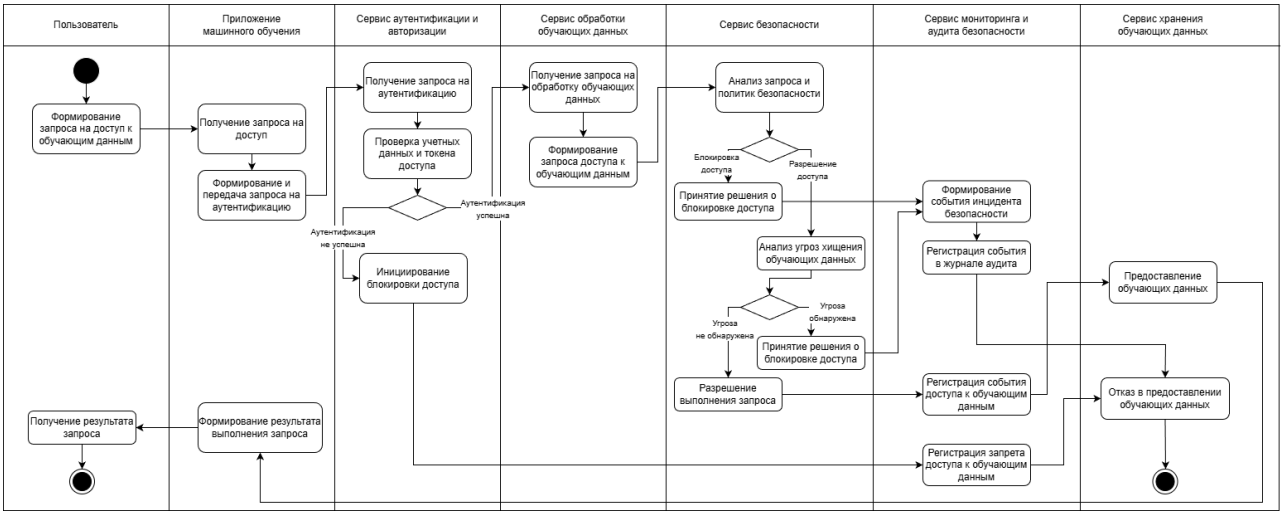


Рисунок 8 – UML-диаграмма деятельности паттерна безопасности

ГЛАВА 3. РАЗРАБОТКА МИКРОСЕРВИСОВ

В рамках реализации разработанного паттерна безопасности была выполнена разработка набора специализированных микросервисов, обеспечивающих защиту приложения машинного обучения от угроз несанкционированного доступа к обучающим данным. Каждый микросервис реализует строго определённую функцию в общей архитектуре и взаимодействует с другими компонентами системы посредством последовательных вызовов.

Микросервисный подход позволяет обеспечить модульность, масштабируемость и отказоустойчивость системы безопасности, а также упростить сопровождение и расширение функциональности паттерна. В данной главе последовательно рассматриваются функции каждого микросервиса, а также логика их работы, проиллюстрированная с помощью блок-схем.

3.1 ФУНКЦИИ МИКРОСЕРВИСОВ

Для реализации разработанного архитектурного паттерна обеспечения безопасности приложения машинного обучения при угрозе хищения обучающих данных были спроектированы и реализованы пять специализированных микросервисов, каждый из которых выполняет строго определённые функции в рамках системы защиты. Все микросервисы реализованы на языке Python и взаимодействуют между собой посредством HTTP API, что соответствует принципам микросервисной архитектуры и обеспечивает слабую связанность компонентов системы.

В качестве базового подхода используется контейнеризация сервисов с возможностью их последующего развёртывания и масштабирования в оркестрируемой среде. Реализация микросервисов ориентирована на обработку запросов приложения машинного обучения и централизованный контроль доступа к обучающим данным.

1. Микросервис аутентификации и авторизации (Authentication and Authorization Service)

Сервис аутентификации и авторизации является ключевым компонентом разработанного архитектурного паттерна обеспечения безопасности

приложения машинного обучения. Его основной задачей является подтверждение подлинности пользователя (или сервиса), а также формирование доверенного контекста доступа, используемого при дальнейшем взаимодействии с обучающими данными.

В рамках данного микросервиса реализованы две основные функциональные группы:

- верификация JWT-токена при поступлении запроса;
- аутентификация пользователя по учетным данным и формирование нового токена доступа.

Для наглядного представления логики работы сервиса каждая из указанных функций описана отдельной блок-схемой.

Первая блок-схема отражает процесс проверки корректности и подлинности JWT-токена, передаваемого в запросе к системе. Данная операция выполняется при каждом обращении к защищённым ресурсам приложения машинного обучения и служит первой линией защиты от несанкционированного доступа.

Процесс начинается с получения JWT access token, содержащего сведения о пользователе и его правах доступа. Далее выполняется декодирование токена и проверка его цифровой подписи с использованием секретного ключа сервиса аутентификации. На этом этапе определяется, был ли токен сформирован доверенным сервисом и не был ли он изменён после выпуска.

В случае если токен является недействительным (подпись не совпадает, истёк срок действия или нарушена структура), формируется ошибка аутентификации, и дальнейшая обработка запроса прекращается. При успешной проверке из токена извлекаются идентификатор пользователя и связанные с ним роли, после чего формируется контекст пользователя, используемый для проверки политик доступа на последующих этапах. Логика описанного процесса представлена на рисунке 9 первой функции сервиса аутентификации.

Фрагмент 1 – Функция верификации JWT-токена

```
def verify_token(self, token: str) -> Optional[Dict]:
        try:
            payload = jwt.decode(
```

```
            token,
            self.secret_key,
            algorithms=[self.algorithm],
            issuer="ml_security_service"
        )
        return payload
    except InvalidTokenError:
        return None
```

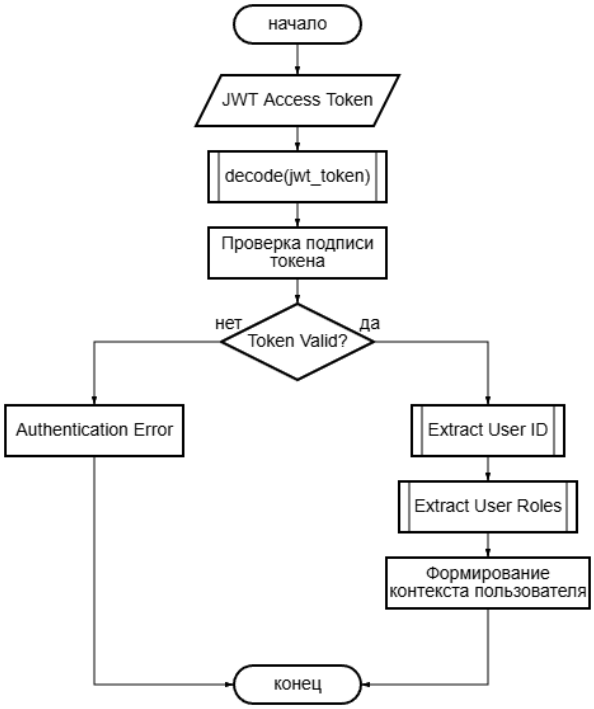


Рисунок 9 – Блок-схема

Вторая блок-схема описывает процесс аутентификации пользователя по логину и паролю и используется при первичном входе пользователя в систему. Результатом выполнения данной функции является формирование нового JWT-токена, который в дальнейшем применяется для доступа к защищённым ресурсам.

Процесс начинается с получения учетных данных пользователя, включающих имя пользователя и пароль. Далее осуществляется запрос к базе данных пользователей с целью проверки существования учетной записи. Если пользователь не найден, процесс аутентификации завершается с ошибкой.

При наличии пользователя выполняется проверка корректности введённого пароля путём сравнения с хэшированным значением, сохранённым

в базе данных. В случае несоответствия пароля формируется ошибка аутентификации. Если проверка проходит успешно, формируется payload JWT-токена, включающий идентификатор пользователя, его роли и разрешения доступа.

На завершающем этапе производится криптографическое подписание токена с использованием секретного ключа, после чего пользователю возвращается access token и сопутствующая информация о его правах доступа. Данный алгоритм представлен на рисунке 10 микросервиса аутентификации.

Фрагмент 2 – Функция аутентификации пользователя и формирования токена доступа

```
def authenticate_user(self, credentials: UserCredentials) -> 
Optional[Dict]:
        user_db = self._get_user_from_db(credentials.username)
        if not user_db:
            return None
        if not self.pwd_context.verify(credentials.password, 
user_db["hashed_password"]):
            return None
        payload = {
            "sub": user_db["username"],
            "roles": user_db["roles"],  # Например: 
["ml_engineer", "data_reader"]
            "permissions": user_db["permissions"],  # 
["read_dataset", "train_model"]
            "iss": "ml_security_service",
            "iat": datetime.utcnow(),
            "exp": datetime.utcnow() + timedelta(hours=1)
        }
        token = jwt.encode(payload, self.secret_key, 
algorithm=self.algorithm)
        return {
            "access_token": token,
            "token_type": "bearer",
            "user_info": {
                "username": user_db["username"],
```

```
            "roles": user_db["roles"],
            "permissions": user_db["permissions"]
        }
    }
```

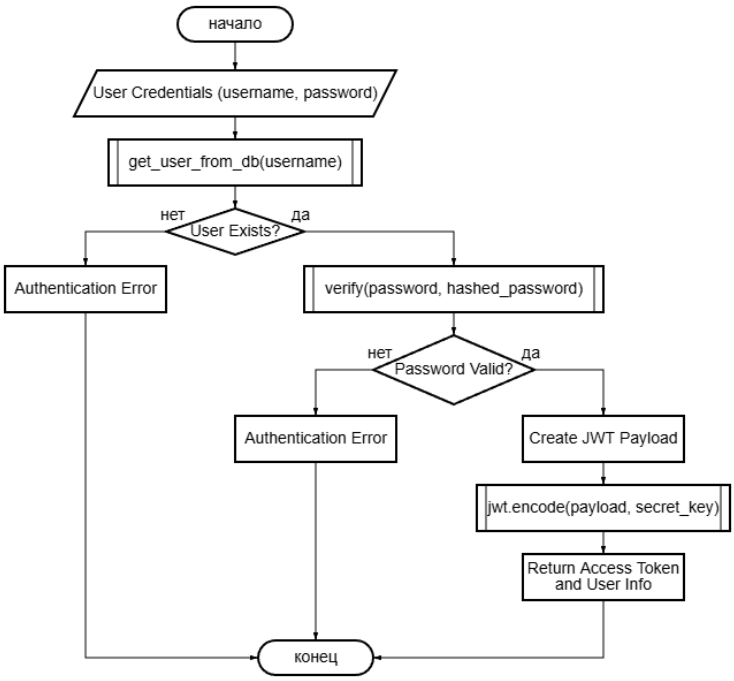


Рисунок 10 – Блок-схема

2. Сервис обработки обучающих данных (Training Data Processing Service)

Блок-схема, представленная на рисунке 11, отражает общий алгоритм функционирования сервиса обработки обучающих данных при выполнении запроса пользователя. Процесс начинается с получения запроса на выполнение операции над обучающим набором данных, содержащего идентификатор набора, тип операции и параметры обработки.

На первом этапе осуществляется проверка прав доступа пользователя посредством обращения к сервису безопасности. В случае отказа в доступе выполнение запроса немедленно прекращается и возвращается соответствующее сообщение об ошибке. Данный подход исключает возможность несанкционированного обращения к обучающим данным.

При успешной проверке прав доступа сервис инициирует получение данных исключительно через сервис безопасности, что предотвращает прямое

взаимодействие с хранилищем данных. После получения данных выполняется проверка корректности их загрузки. В случае ошибки получения данных выполнение запроса завершается с возвратом сообщения об ошибке.

Если данные успешно получены, сервис выполняет запрошенную операцию обработки, после чего фиксирует факт успешного выполнения операции и формирует результат обработки. Завершением процесса является возврат результата обработки обучающих данных.

Фрагмент 3 – Функция обработки запроса к обучающим данным

```
def process_dataset(self, request: DatasetRequest) ->
Optional[Dict]:
    if not self._validate_access(request):
        return None
    raw_data = self._fetch_data_through_security(request)
    if raw_data is None:
        return None
    processed_data = self._execute_operation(
        raw_data, request.operation, request.parameters
    )
    self._log_operation_success(request, ...)
    return {...}
```

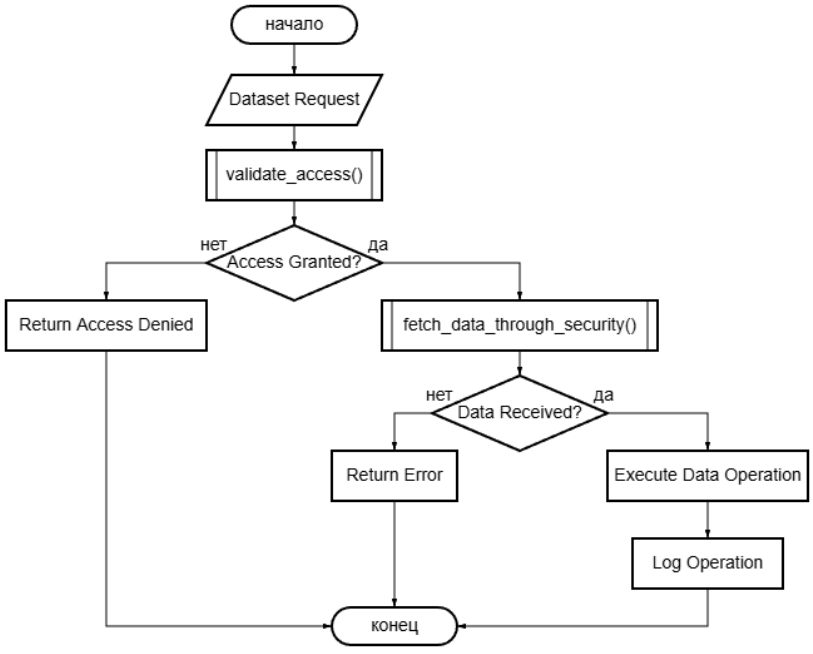


Рисунок 11 – Блок-схема

Блок-схема, представленная на рисунке 12, детализирует внутреннюю логику выполнения операций обработки обучающих данных в зависимости от типа запроса. Процесс начинается с определения типа операции, переданного в параметрах запроса.

Далее осуществляется последовательная проверка поддерживаемых типов операций: фильтрация данных, выборка, аугментация и предобработка. Для каждого типа операции предусмотрена отдельная ветвь выполнения, в рамках которой вызывается соответствующий алгоритм обработки данных.

В случае, если тип операции не соответствует ни одному из поддерживаемых вариантов, выполнение запроса завершается с формированием ошибки некорректной операции. Такой механизм обеспечивает защиту сервиса от некорректных или потенциально опасных запросов.

При успешном выполнении одной из поддерживаемых операций процесс обработки данных завершается, и управление возвращается в основной алгоритм сервиса для дальнейшей обработки результатов и фиксации события.

Фрагмент 4 – Функция выбора и выполнения операции обработки данных

```
def _execute_operation(self, data, operation, parameters):
    if operation == "filter":
        return self._filter_data(data, parameters)
    elif operation == "sample":
        return self._sample_data(data, parameters)
    elif operation == "augment":
        return self._augment_data(data, parameters)
    elif operation == "preprocess":
        return self._preprocess_data(data, parameters)
    else:
        raise ValueError("Unknown operation")
```

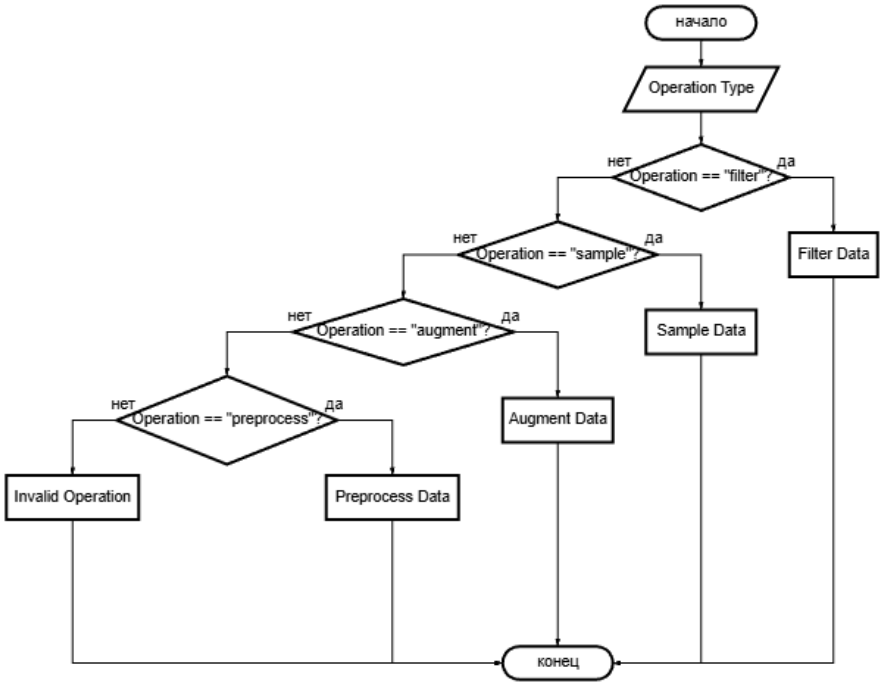


Рисунок 12 – Блок-схема

3. Сервис безопасности (Security Evaluation Service)

На рисунке 13 представлена блок-схема основного алгоритма работы сервиса безопасности, реализующего оценку запроса на доступ к обучающим данным.

Процесс начинается с поступления запроса безопасности, содержащего идентификаторы пользователя и набора данных, тип операции, токен доступа и контекст выполнения. На первом этапе осуществляется проверка наличия ранее принятого решения в кэше. При обнаружении закэшированного решения сервис немедленно возвращает его без выполнения дополнительных проверок, что позволяет снизить нагрузку на систему и повысить производительность.

В случае отсутствия закэшированного решения выполняется последовательность многоуровневых проверок безопасности, включающая валидацию JWT-токена, анализ политик доступа, проверку контекста запроса, выявление индикаторов угроз и контроль частоты обращений. После завершения всех проверок производится анализ их результатов и определяется уровень угрозы.

Если уровень угрозы признаётся критическим, принимается решение об отказе в доступе. В противном случае формируется итоговое решение

безопасности, которое может быть сохранено в кэше и зафиксировано в журнале аудита. Процесс завершается возвратом решения сервису-инициатору запроса.

Фрагмент 5 – Основной метод оценки запроса на доступ

```
async def evaluate_access_request(self, request: SecurityRequest)
-> SecurityResponse:
    start_time = datetime.utcnow()
    cache_key = self._generate_cache_key(request)
    cached_decision = await self._get_cached_decision(cache_key)
    if cached_decision:
        return cached_decision
    checks = [
        self._validate_token(request.token),
        self._check_access_policies(request.user_id,
request.dataset_id, request.operation),
        self._analyze_request_context(request),
        await self._check_threat_indicators(request),
        self._verify_rate_limits(request.user_id,
request.ip_address)
    ]
    check_results = await asyncio.gather(*checks,
return_exceptions=True)
    decision, reasons, threat_level =
self._analyze_check_results(check_results)
    if threat_level == "critical":
        decision = SecurityDecision.DENY
    response = SecurityResponse(
        decision=decision,
        access_granted=decision == SecurityDecision.ALLOW,
        reasons=reasons,
        threat_level=threat_level
    )
    if threat_level != "critical":
        await self._cache_decision(cache_key, response, ttl=300)
    await self._audit_decision(request, response, start_time)
    return response
```

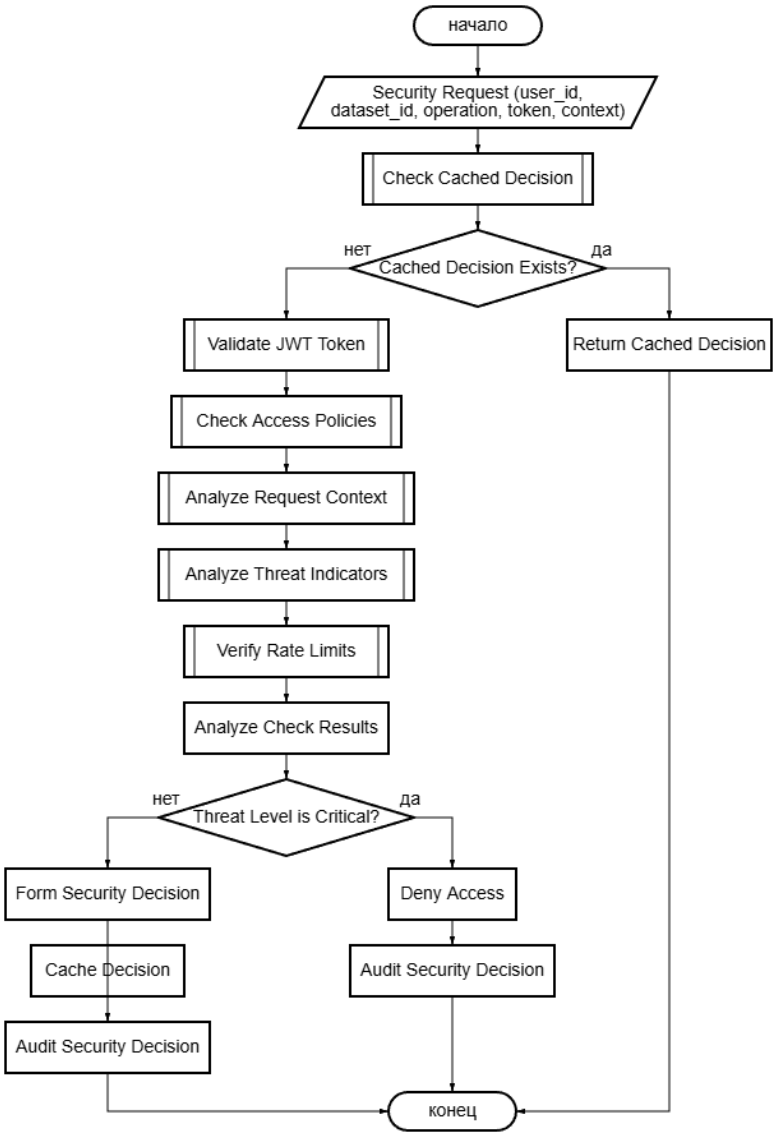


Рисунок 13 – Блок-схема

На рисунке 14 представлена блок-схема анализа результатов проверок безопасности, выполняемого в рамках сервиса безопасности. Данный алгоритм предназначен для формирования итогового решения о предоставлении или блокировке доступа к обучающим данным.

Процесс начинается с получения результатов всех ранее выполненных проверок, а также информации об уровне выявленных угроз. На этапе анализа результатов производится оценка количества неуспешных проверок и степени потенциальной угрозы.

В случае обнаружения критического уровня угрозы принимается решение о немедленном отказе в доступе, после чего выполняется аудит принятого

решения. Если критические угрозы отсутствуют, формируется итоговое решение безопасности, которое может предусматривать разрешение доступа, необходимость дополнительной проверки или отказ.

Принятое решение кэшируется для оптимизации повторных запросов и регистрируется в журнале аудита безопасности. Завершением процесса является фиксация результата и возврат решения в рамках архитектуры паттерна.

Фрагмент 6 – Логика анализа результатов проверок безопасности

```
def _analyze_check_results(self, check_results: List) ->
Tuple[SecurityDecision, List[str], str]:
    reasons = []
    failed_checks = 0
    threat_level = "low"
    for result in check_results:
        if isinstance(result, Exception):
            reasons.append(str(result))
            failed_checks += 1
            continue
        if isinstance(result, tuple):
            if len(result) == 3:
                passed, reason, level = result
                if level in ["medium", "high", "critical"]:
                    threat_level = level
            else:
                passed, reason = result
            if not passed:
                reasons.append(reason)
                failed_checks += 1
    if threat_level == "critical" or failed_checks >= 2:
        decision = SecurityDecision.DENY
    elif failed_checks == 1 or threat_level in ["medium", "high"]:
        decision = SecurityDecision.REVIEW
    else:
        decision = SecurityDecision.ALLOW
    return decision, reasons, threat_level
```

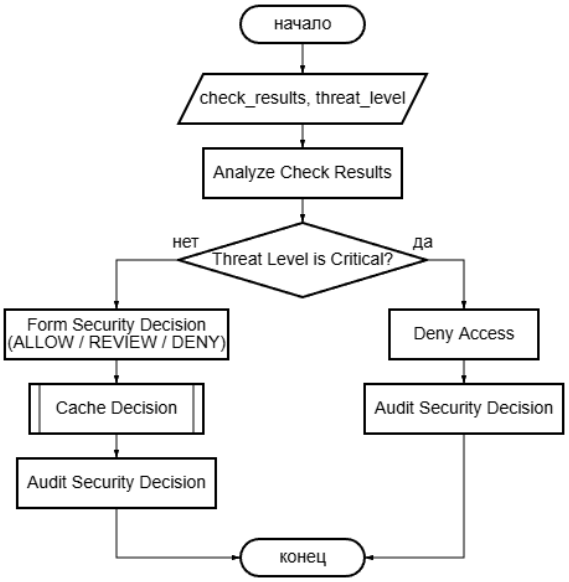


Рисунок 14 – Блок-схема

4. Сервис мониторинга и аудита безопасности (Security Monitoring and Audit Service)

На рисунке 15 представлена блок-схема логики работы микросервиса мониторинга и аудита безопасности при поступлении события безопасности от других компонентов системы. Процесс начинается с получения события аудита, содержащего идентификатор события, тип, пользователя, выполняемое действие и временную метку. На первом этапе осуществляется проверка целостности события путём валидации хеша, что позволяет гарантировать неизменность данных аудита и предотвратить подмену или искажение информации.

В случае успешной проверки целостности событие сохраняется в централизованное хранилище журналов безопасности на базе Elasticsearch. После этого выполняется обновление статистики в реальном времени, которая используется для анализа активности пользователей и выявления подозрительных паттернов поведения. Далее запускается процедура обнаружения аномалий, направленная на выявление отклонений от нормального поведения системы. Независимо от результата анализа событие дополнительно архивируется для долгосрочного хранения, что обеспечивает сохранность истории инцидентов и возможность последующего расследования. При

нарушении целостности события его обработка прекращается, и событие отклоняется без сохранения в журнале аудита.

Фрагмент 7 – Логирование события безопасности и проверка целостности

```
async def log_event(self, event: AuditEvent) -> bool:
    try:
        if not self._verify_integrity_hash(event):
            return False
        await self.es_client.index(
            index=self.events_index,
            document=event.dict(),
            id=event.event_id
        )
        await self._update_realtime_stats(event)
        await self._detect_anomalies(event)
        await self._archive_event(event)
        return True
    except Exception:
        return False
```

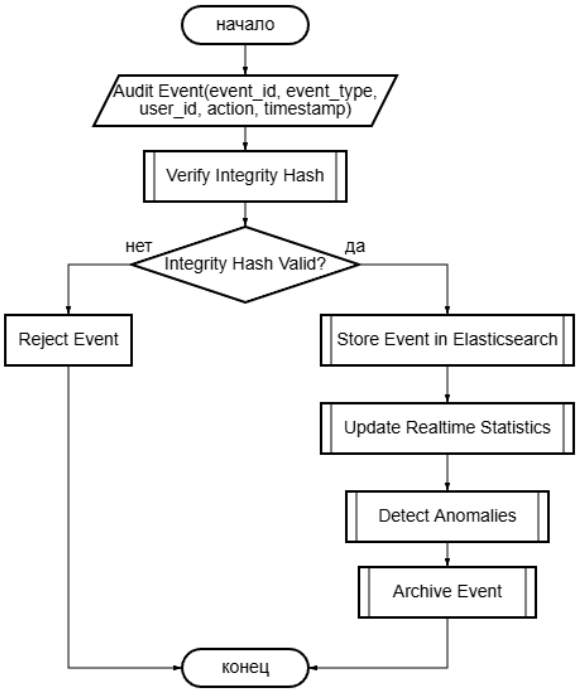


Рисунок 15 – Блок-схема

На рисунке 16 представлена блок-схема, которая описывает процесс анализа проверенного события безопасности на наличие признаков аномальной

активности. Процесс начинается с получения события аудита, прошедшего проверку целостности. Далее событие сопоставляется с набором заранее определённых шаблонов аномалий, включающих превышение допустимой частоты запросов, доступ к данным вне рабочего времени, множественные неудачные попытки аутентификации и массовый экспорт чувствительных данных.

Если событие не соответствует ни одному из шаблонов аномалий, дополнительных действий не требуется, и процесс завершается. В случае обнаружения аномалии определяется уровень её серьёзности, после чего формируется оповещение безопасности. Созданное оповещение сохраняется в хранилище оповещений Elasticsearch для последующего анализа и контроля. При критическом уровне угрозы дополнительно инициируется отправка уведомлений администраторам безопасности с целью оперативного реагирования на инцидент. Такой подход позволяет своевременно выявлять потенциальные попытки хищения обучающих данных и минимизировать возможный ущерб.

Фрагмент 8 – Обнаружение аномалий и генерация оповещений безопасности

```
async def _detect_anomalies(self, event: AuditEvent):
    detected_anomalies = []
    for pattern in self.anomaly_patterns:
        if self._matches_anomaly_pattern(event, pattern):
            detected_anomalies.append(pattern)
    if detected_anomalies:
        await self._create_alert_from_anomalies(event,
detected_anomalies)
def _matches_anomaly_pattern(
    self,
    event: AuditEvent,
    pattern: AnomalyPattern
) -> bool:
    if pattern.pattern_name == "burst_requests":
        stats = self.realtime_stats.get(event.user_id, {})
        return stats.get("events_count", 0) > pattern.threshold
```

```
    if pattern.pattern_name == "unusual_time_access":
        hour = event.timestamp.hour
        return hour < 6 or hour > 22
    if pattern.pattern_name == "multiple_failed_auth":
        stats = self.realtime_stats.get(event.user_id, {})
        return stats.get("failed_auth", 0) > pattern.threshold
    if pattern.pattern_name == "sensitive_data_export":
        if event.event_type == EventType.DATA_ACCESS:
            return event.details.get("record_count", 0) >
pattern.threshold
    return False
async def _create_alert_from_anomalies(
    self,
    event: AuditEvent,
    anomalies: List[AnomalyPattern]
):
    max_severity = max(
        anomalies,
        key=lambda x: list(EventSeverity).index(x.severity)
    )
    alert = SecurityAlert(
alert_id=f"alert_{datetime.utcnow().strftime('%Y%m%d_%H%M%S_%f')}"
,
        alert_type="anomaly_detection",
        severity=max_severity.severity,
        title="Detected anomalous activity",
        description=f"Suspicious action detected: {event.action}",
        affected_resources=[event.resource_id] if
event.resource_id else []
    )
    await self.es_client.index(
        index=self.alerts_index,
        document=alert.dict(),
        id=alert.alert_id
    )
    if alert.severity in [
        EventSeverity.ERROR,
        EventSeverity.CRITICAL
    ]:
```

```
        await self._send_notifications(alert)
```

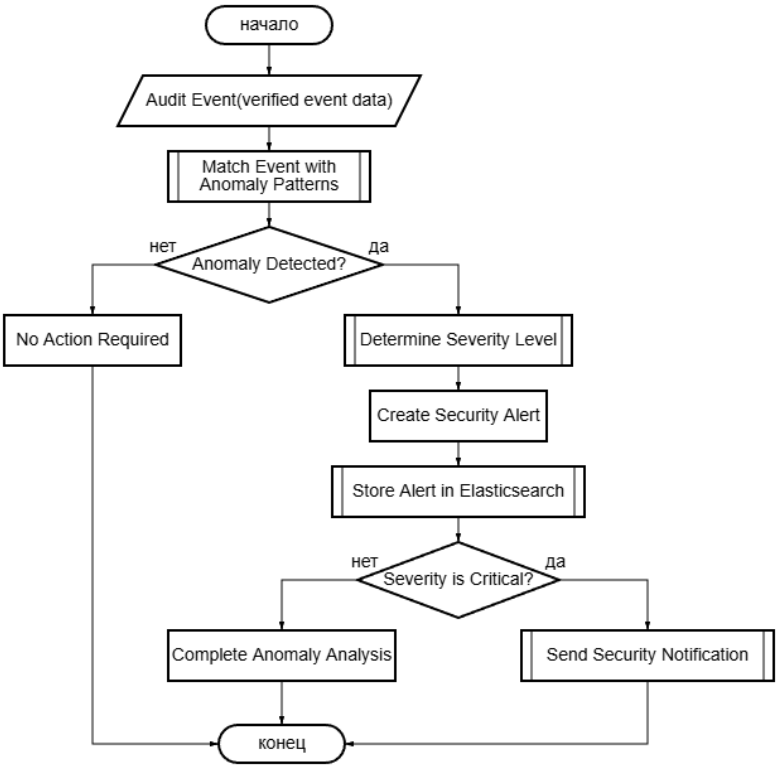


Рисунок 16 – Блок-схема

На рисунке 17 представлена блок-схема, отражающая процесс формирования аналитического отчёта по безопасности на основе накопленных данных аудита. Процесс начинается с получения запроса на формирование отчёта за заданный временной период. На первом этапе осуществляется выборка событий безопасности из хранилища Elasticsearch с учётом указанных временных границ.

После получения данных выполняется их агрегирование и аналитическая обработка, включающая подсчёт общего количества событий, анализ активности пользователей, выявление наиболее часто используемых ресурсов, оценку частоты нарушений политик доступа и анализ уровней серьёзности инцидентов. На основе полученных результатов формируется комплексный отчёт, содержащий сводную статистику, показатели риска, информацию о выявленных аномалиях и рекомендации по повышению уровня безопасности. Сформированный отчёт возвращается инициатору запроса и может

использоваться для оценки эффективности реализованных механизмов защиты и принятия управленческих решений.

Фрагмент 9 – Формирование аналитического отчета по событиям безопасности

```
async def generate_security_report(
    self,
    start_date: datetime,
    end_date: datetime
) -> Dict:
    query = {
        "query": {
            "range": {
                "timestamp": {
                    "gte": start_date.isoformat(),
                    "lte": end_date.isoformat()
                }
            }
        }
    }
    response = await self.es_client.search(
        index=self.events_index,
        body=query,
        size=10000
    )
    events = [
        hit["_source"]
        for hit in response["hits"]["hits"]
    ]
    if not events:
        return {
            "message": "No security events found for the selected
period"
        }
    df = pd.DataFrame(events)
    report = {
        "period": {
            "start": start_date.isoformat(),
```

```
            "end": end_date.isoformat()
        },
        "summary": {
            "total_events": len(df),
            "unique_users": df["user_id"].nunique(),
            "events_by_type":
df["event_type"].value_counts().to_dict(),
            "events_by_severity":
df["severity"].value_counts().to_dict()
        }
    }
    return report
```

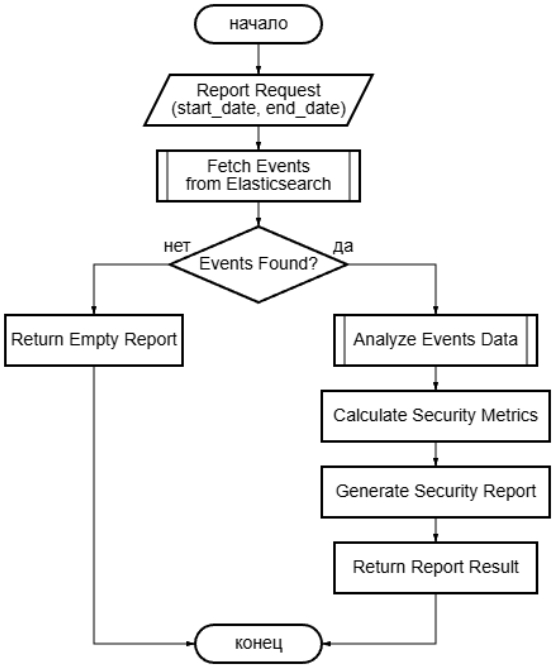


Рисунок 17 – Блок-схема

5. Сервис хранения обучающих данных (Training Data Storage Service)

Рисунок 18 иллюстрирует блок-схему проверки прав доступа пользователя к операциям с обучающими данными в сервисе хранения. Процесс начинается с получения контекста безопасности и запрашиваемой операции. На первом этапе выполняется проверка наличия разрешения на доступ к ресурсу. В случае отсутствия прав доступ немедленно отклоняется.

При успешной проверке прав доступа осуществляется дополнительная проверка допустимости запрашиваемой операции. Если операция не разрешена действующими политиками безопасности, доступ отклоняется. В противном

случае формируется положительное решение о предоставлении доступа, после чего процесс завершается. Данная схема обеспечивает строгий контроль допустимых операций с обучающими данными и предотвращает несанкционированные действия.

Фрагмент 10 – Проверка контекста безопасности запроса

```
def validate_security_context(
    security_context: Dict,
    operation: str
) -> bool:
    if not security_context.get("access_granted", False):
        return False
    allowed_operations =
security_context.get("allowed_operations", [])
    if operation not in allowed_operations:
        return False
    return True
```

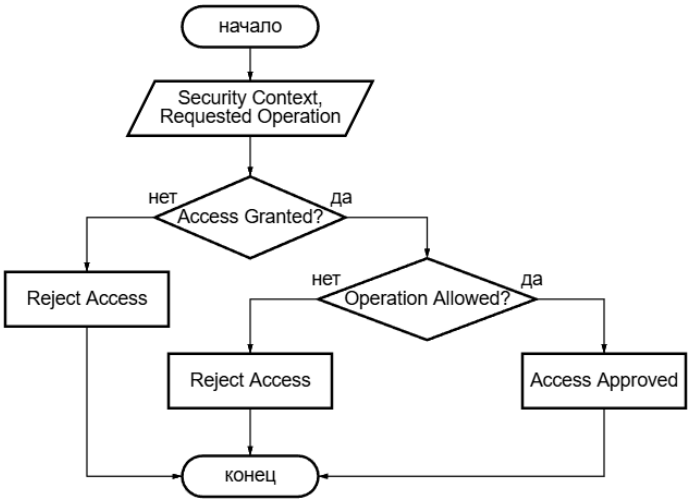


Рисунок 18 – Блок-схема

Рисунок 19 представляет блок-схему процесса чтения обучающего набора данных из хранилища. Процесс начинается с получения идентификатора набора данных, после чего выполняется загрузка соответствующего объекта из системы хранения.

На следующем этапе осуществляется проверка наличия запрашиваемого набора данных. В случае отсутствия данных формируется ответ об ошибке с

указанием, что набор данных не найден. Если набор данных успешно загружен, вычисляется контрольная сумма для проверки целостности информации, после чего подготавливается ответ с данными набора и сопутствующей метаинформацией. Процесс завершается возвратом результата операции чтения.

Фрагмент 11 – Чтение обучающих данных из хранилища

```
async def read_dataset(dataset_id: str):
    object_name = f"{dataset_id}.json"
    response = storage_client.get_object(
        BUCKET_NAME,
        object_name
    )
    data = json.loads(response.read().decode())
    checksum = generate_checksum(data)
    return {
        "dataset_id": dataset_id,
        "data": data,
        "checksum": checksum
    }
```

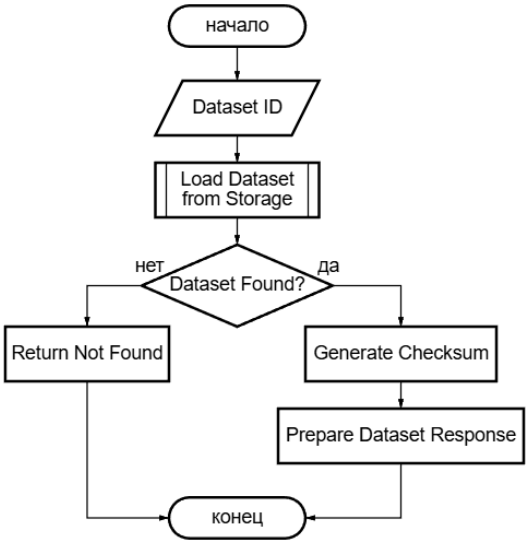


Рисунок 19 – Блок-схема

Рисунок 20 отображает блок-схему процесса записи обучающего набора данных в систему хранения. Процесс начинается с получения идентификатора набора данных, самих данных и метаданных. Далее выполняется расчет

контрольной суммы, обеспечивающей контроль целостности сохраняемой информации.

После этого формируется объект набора данных, включающий данные, метаданные, контрольную сумму и информацию о версии. Подготовленный объект передается в систему хранения для сохранения. По завершении операции формируется результат выполнения записи, который возвращается вызывающей стороне, после чего процесс завершается.

Фрагмент 12 – Запись и обновление обучающих данных

```
async def write_dataset(
    dataset_id: str,
    data: Dict,
    metadata: Dict
):
    checksum = generate_checksum(data)
    object_name = f"{dataset_id}.json"
    dataset_object = {
        "dataset_id": dataset_id,
        "data": data,
        "metadata": metadata,
        "checksum": checksum,
        "version": 1
    }
    storage_client.put_object(
        BUCKET_NAME,
        object_name,
        data=json.dumps(dataset_object).encode(),
        length=len(json.dumps(dataset_object))
    )
    return {
        "dataset_id": dataset_id,
        "checksum": checksum
    }
```

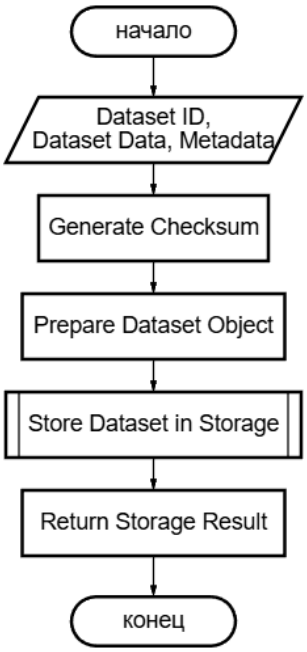


Рисунок 20 – Блок-схема

3.2 РАЗВЕРТЫВАНИЕ МИКРОСЕРВИСА

Развёртывание разработанных микросервисов паттерна безопасности осуществляется с использованием технологий Docker и Kubernetes, что обеспечивает изоляцию окружения, масштабируемость и автоматическое восстановление при сбоях.

Контейнеризация позволяет запускать каждый компонент независимо — микросервисы аутентификации, обработки данных, безопасности, мониторинга и хранения данных работают как автономные контейнеры, объединённые в общую сеть безопасности.

На этапе локального тестирования применяется docker-compose, где задаются связи между сервисами, параметры базы данных PostgreSQL и кэширующего хранилища Redis.

После успешной проверки система переносится в кластер Kubernetes, где для каждого микросервиса создаются объекты Deployment и Service, а общие параметры безопасности задаются через Secret и ConfigMap.

Постоянные данные (логи, политики, учётные записи, обучающие данные) хранятся в PersistentVolume, что обеспечивает сохранность информации даже при перезапуске контейнеров.

Такой подход упрощает обновление компонентов и повышает отказоустойчивость. Kubernetes выполняет балансировку нагрузки и автоматическое масштабирование при росте числа запросов, а система мониторинга Prometheus и панель Grafana предоставляют непрерывный контроль метрик и состояния всех микросервисов.

Инфраструктура построена по принципам DevOps: сборка и доставка контейнеров выполняются автоматически, что сокращает время развертывания и снижает вероятность ошибок конфигурации.

Перед началом работы необходимо установить Docker Desktop на компьютер. После установки убедимся, что Docker запущен, проверив его работу командой docker --version в терминале. Далее нужно создать рабочую директорию для проекта, например, ml_security_system, и внутри неё организовать структуру папок для каждого микросервиса:

- auth-service
- data-processing-service
- security-service
- monitoring-service
- storage-service

В каждой папке микросервиса необходимо разместить соответствующий Dockerfile и исходный код Python-приложения.

Создадим файл *docker-compose.yaml*, где будет описана вся инфраструктура безопасности. Конфигурация будет включать следующие сервисы:

- микросервис аутентификации и авторизации на порте 8081
- микросервис обработки обучающих данных на порте 8082
- микросервис безопасности на порте 8083
- микросервис мониторинга и аудита на порте 8084
- микросервис хранения обучающих данных на порте 8085

Каждый сервис имеет индивидуальные настройки ресурсов (ограничения CPU и памяти), переменные окружения для подключения к базе данных PostgreSQL, а также зависимости от других сервисов. Для обеспечения

отказоустойчивости настроены политики перезапуска контейнеров при сбоях, а также health-чеки для мониторинга работоспособности каждого микросервиса. Сетевой стек организован в виде изолированной сети security-net, что предотвращает несанкционированный доступ к внутренним коммуникациям между микросервисами.

1. Микросервис аутентификации и авторизации

```
auth-service:
  build: ./auth-service
  ports:
    - "8081:80"
  environment:
    -
DATABASE_URL=postgresql://user:password@postgres:5432/security_db
    - REDIS_URL=redis://redis:6379
    - JWT_SECRET=${JWT_SECRET}
  networks:
    - security-net
  depends_on:
    - postgres
    - redis
```

2. Микросервис обработки обучающих данных

```
data-processing-service:
  build: ./data-processing-service
  ports:
    - "8082:80"
  environment:
    -
DATABASE_URL=postgresql://user:password@postgres:5432/security_db
    - SECURITY_SERVICE_URL=http://security-service:80
  networks:
    - security-net
  depends_on:
    - postgres
    - security-service
```

3. Микросервис безопасности

```
security-service:
```

```
  build: ./security-service
  ports:
    - "8083:80"
  environment:
    -
DATABASE_URL=postgresql://user:password@postgres:5432/security_db
    - REDIS_URL=redis://redis:6379
  networks:
    - security-net
  depends_on:
    - postgres
    - redis
```

4. Микросервис мониторинга и аудита безопасности

```
monitoring-service:
  build: ./monitoring-service
  ports:
    - "8084:80"
  environment:
    -
DATABASE_URL=postgresql://user:password@postgres:5432/security_db
    - ELASTICSEARCH_URL=http://elasticsearch:9200
  networks:
    - security-net
  depends_on:
    - postgres
    - elasticsearch
```

5. Микросервис хранения обучающих данных

```
storage-service:
  build: ./storage-service
  ports:
    - "8085:80"
  environment:
    -
DATABASE_URL=postgresql://user:password@postgres:5432/security_db
    - MINIO_ENDPOINT=minio:9000
    - MINIO_ACCESS_KEY=${MINIO_ACCESS_KEY}
    - MINIO_SECRET_KEY=${MINIO_SECRET_KEY}
  networks:
```

```
    - security-net
  depends_on:
    - postgres
    - minio
```

6. Вспомогательные сервисы (СУБД, кэш, хранилище)

```
postgres:
  image: postgres:13
  environment:
    - POSTGRES_DB=security_db
    - POSTGRES_USER=user
    - POSTGRES_PASSWORD=password
  volumes:
    - postgres_data:/var/lib/postgresql/data
  networks:
    - security-net
redis:
  image: redis:alpine
  networks:
    - security-net

elasticsearch:
  image: elasticsearch:7.17.0
  environment:
    - discovery.type=single-node
  volumes:
    - es_data:/usr/share/elasticsearch/data
  networks:
    - security-net

minio:
  image: minio/minio
  command: server /data
  environment:
    - MINIO_ROOT_USER=${MINIO_ACCESS_KEY}
    - MINIO_ROOT_PASSWORD=${MINIO_SECRET_KEY}
  volumes:
    - minio_data:/data
```

```
  networks:
    - security-net
```

7. Сети и тома

```
volumes:
  postgres_data:
  es_data:
  minio_data:

networks:
  security-net:
    driver: bridge
```

Для развертывания разработанной системы безопасности в контейнеризированной среде Docker в терминале выполняется последовательность команд. На первом этапе необходимо создать файл *.env*, содержащий секретные ключи и параметры доступа, используемые микросервисами:

```
JWT_SECRET=your_super_secret_jwt_key
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin
```

Данный файл позволяет вынести конфиденциальные параметры из конфигурационных файлов и повысить уровень безопасности системы.

После этого выполняется сборка Docker-образов всех микросервисов. Для обеспечения чистой сборки без использования кэшированных слоёв применяется следующая команда:

```
docker-compose build --no-cache
```

Запуск системы осуществляется с помощью команды, развертывающей все микросервисы в фоновом режиме:

```
docker-compose up -d
```

Для проверки состояния контейнеров используется команда, отображающая статус каждого микросервиса:

```
docker-compose ps
```

В процессе эксплуатации системы может потребоваться анализ журналов работы отдельных компонентов. Для этого используется команда просмотра логов конкретного микросервиса, например сервиса аутентификации и авторизации:

```
docker-compose logs -f auth-service
```

Остановка системы выполняется с помощью команды, которая корректно завершает работу всех контейнеров с сохранением данных в подключенном томах:

```
docker-compose down
```

После локального тестирования система переносится в кластер Kubernetes для обеспечения высокой доступности и масштабируемости.

На первом этапе создается отдельное пространство имен для компонентов системы безопасности:

```
kubectl create namespace ml-security
```

Далее создаются секреты для хранения параметров подключения к базе данных и сервису хранения MinIO:

```
kubectl create secret generic db-secret \
  --from-literal=database-
url=postgresql://user:password@postgres:5432/security_db \
  -n ml-security


kubectl create secret generic minio-secret \
  --from-literal=access-key=${MINIO_ACCESS_KEY} \
  --from-literal=secret-key=${MINIO_SECRET_KEY} \
  -n ml-security
```

Для обеспечения сохранности данных применяются манифесты постоянных томов хранения:

```
kubectl apply -f postgres-pv.yaml -n ml-security
```

```
kubectl apply -f minio-pv.yaml -n ml-security
```

После этого выполняется развертывание микросервисов архитектурного паттерна безопасности:

```
kubectl apply -f auth-service-deployment.yaml -n ml-security
kubectl apply -f data-processing-service-deployment.yaml -n ml-security
kubectl apply -f security-service-deployment.yaml -n ml-security
kubectl apply -f monitoring-service-deployment.yaml -n ml-security
kubectl apply -f storage-service-deployment.yaml -n ml-security
```

Для организации сетевого доступа к микросервисам применяются соответствующие Kubernetes-сервисы:

```
kubectl apply -f services.yaml -n ml-security
```

Контроль состояния развернутых компонентов осуществляется с помощью команды:

```
kubectl get all -n ml-security
```

В целях повышения отказоустойчивости и производительности для сервиса безопасности настраивается автоматическое масштабирование в зависимости от загрузки процессора:

```
kubectl autoscale deployment security-service \
  --cpu-percent=70 \
  --min=2 \
  --max=5 \
  -n ml-security
```

4. ТЕСТИРОВАНИЕ И МОНИТОРИНГ МИКРОСЕРВИСОВ НА КЛАСТЕРЕ

Практическая часть курсовой работы посвящена экспериментальному исследованию разработанного паттерна обеспечения безопасности приложения при угрозе хищения обучающих данных. В данной главе проводится комплексное тестирование микросервисной архитектуры, а также анализ функционирования механизмов мониторинга и аудита безопасности в контейнеризированной среде.

Целью главы является подтверждение работоспособности разработанной системы в условиях, приближенных к реальной эксплуатации, а также оценка устойчивости микросервисов к несанкционированному доступу и потенциальным угрозам безопасности. Для достижения поставленной цели решаются следующие задачи:

- проверка корректности развертывания всех микросервисов;
- тестирование доступности и базовой функциональности сервисов;
- экспериментальная проверка защитных механизмов;
- мониторинг ключевых метрик производительности и стабильности системы;
- анализ полученных результатов и формирование выводов о надежности архитектуры.

Тестирование и мониторинг выполняются с использованием инструментов Docker Desktop, PowerShell-скриптов, а также систем сбора и визуализации метрик Prometheus и Grafana. Такой подход позволяет комплексно оценить как функциональные, так и нефункциональные характеристики разработанной системы безопасности, включая ее отказоустойчивость, масштабируемость и способность выявлять потенциальные угрозы хищения обучающих данных.

4.1 ТЕСТИРОВАНИЕ МИКРОСЕРВИСОВ

Тестирование микросервисов направлено на экспериментальную проверку корректности функционирования разработанного паттерна

безопасности и подтверждение его способности предотвращать несанкционированный доступ к обучающим данным. В рамках данного этапа проверяется как базовая работоспособность отдельных микросервисов, так и согласованность их взаимодействия в составе единой микросервисной архитектуры.

Основной задачей тестирования является подтверждение того, что все обращения к обучающим данным проходят через централизованный сервис безопасности, а попытки обхода механизмов контроля доступа корректно выявляются и блокируются. Дополнительно оценивается корректность реализации ролевой модели доступа, механизмов обнаружения аномальной активности и регистрации событий безопасности.

Тестирование проводится в контейнеризированной среде с использованием Docker Compose и включает в себя последовательную проверку развертывания сервисов, автоматизированную валидацию их доступности, а также выполнение функциональных тестов, имитирующих потенциальные сценарии хищения обучающих данных.

Первым этапом эксперимента является проверка успешного развертывания всех компонентов системы в контейнеризированной среде Docker. Для этого используется Docker Compose, обеспечивающий автоматизированный запуск микросервисов и их взаимодействие в изолированной сети безопасности.

В интерфейсе Docker Desktop отображаются все контейнеры разработанной системы, включая основные микросервисы безопасности и вспомогательные компоненты мониторинга. Все контейнеры находятся в состоянии Running, что свидетельствует о корректной сборке Docker-образов, отсутствии ошибок конфигурации и успешной инициализации сервисов.

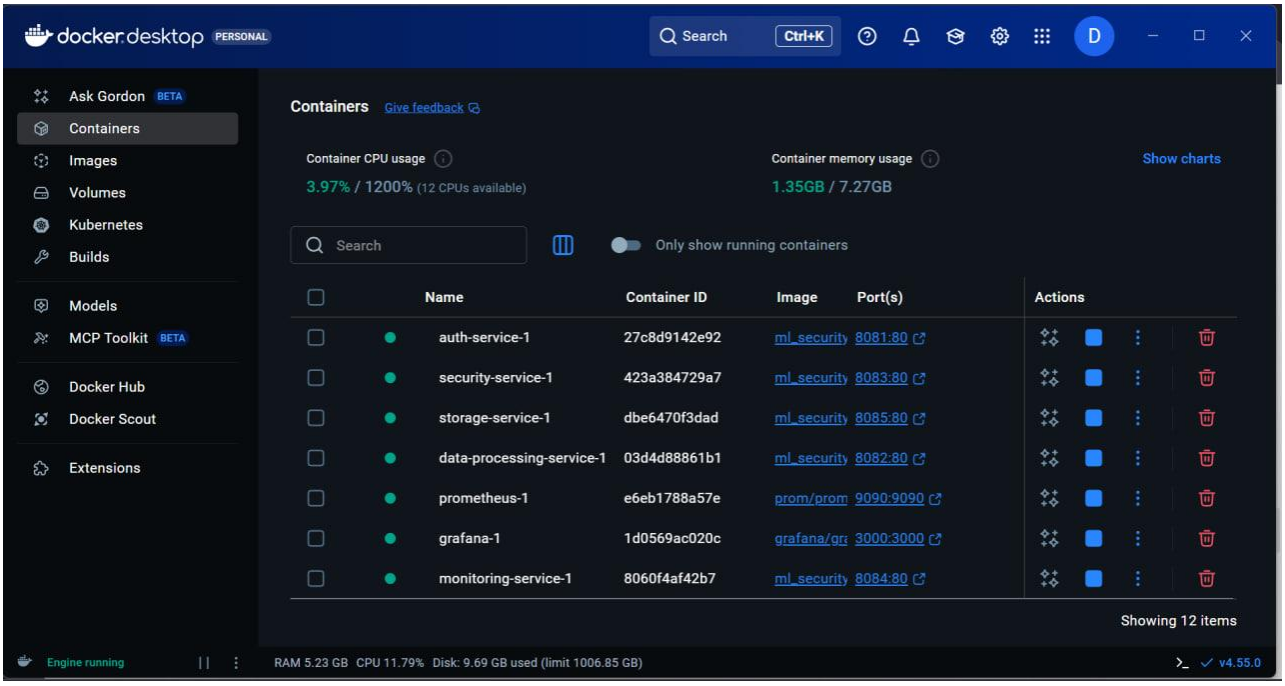


Рисунок 21 — Состояние контейнеров микросервисов системы безопасности в Docker Desktop

Данный этап подтверждает готовность системы к проведению функционального тестирования и дальнейших экспериментов.

Для автоматизированной проверки работоспособности микросервисов был разработан PowerShell-скрипт, который последовательно отправляет HTTP-запросы к эндпоинтам /health каждого сервиса. Такой подход позволяет убедиться, что каждый компонент системы корректно принимает запросы и готов к выполнению своих функций.

Скрипт проверяет следующие сервисы:

- сервис аутентификации и авторизации;
- сервис обработки обучающих данных;
- сервис безопасности;
- сервис мониторинга и аудита;
- сервис хранения данных;
- вспомогательные сервисы Elasticsearch и MinIO.

Результаты выполнения скрипта подтверждают, что все микросервисы возвращают статус healthy, что свидетельствует об их полной работоспособности и корректной интеграции в общую архитектуру.

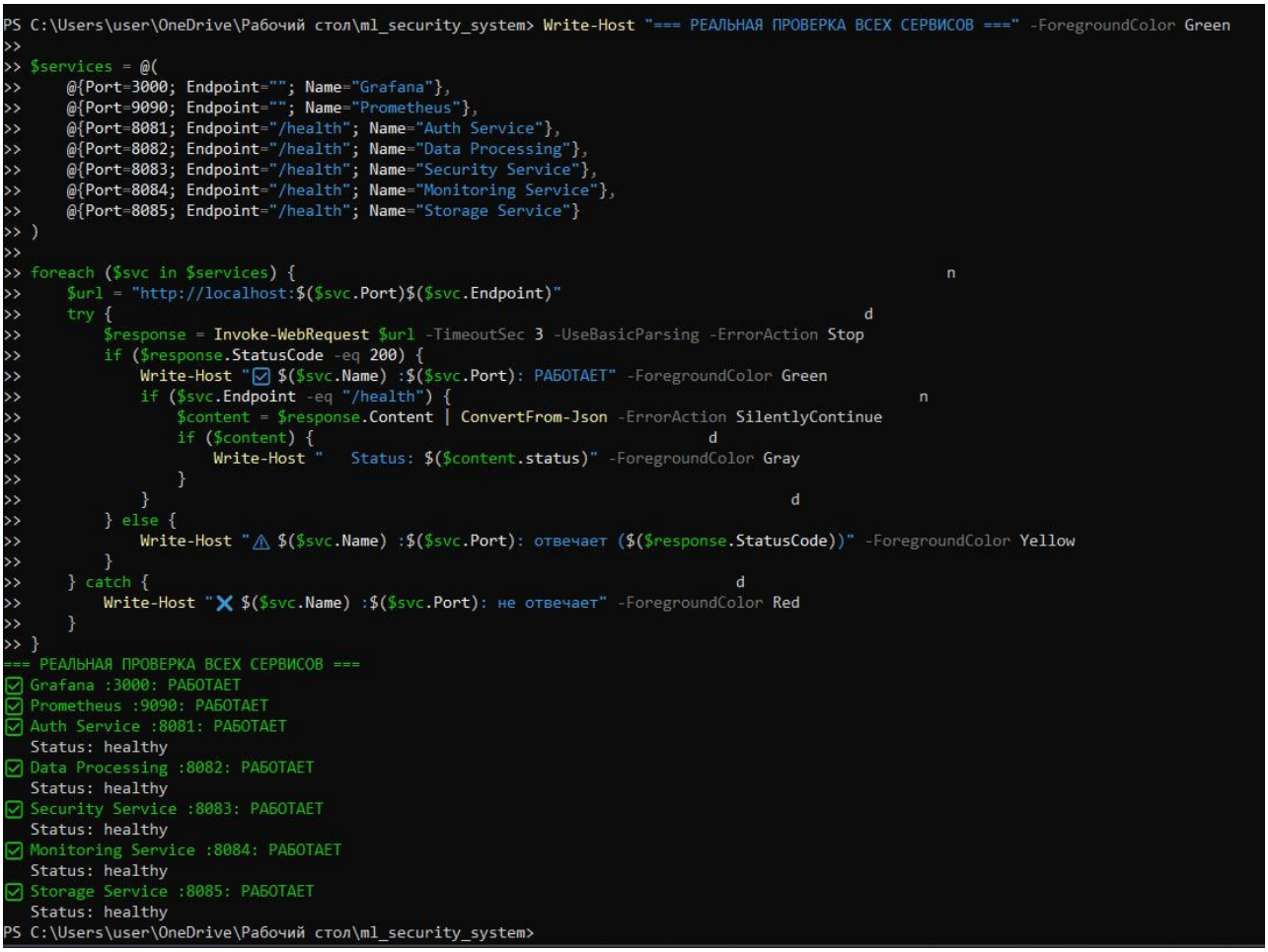


Рисунок 22 — Результаты автоматизированной проверки доступности микросервисов в PowerShell

Для проверки механизмов защиты от хищения обучающих данных был смоделирован сценарий аномальной активности, связанный с массовой выгрузкой данных. В рамках теста выполнялась серия однотипных запросов, имитирующих попытку несанкционированного экспорта большого объема информации.

Система безопасности корректно зафиксировала аномальное поведение и заблокировала все подозрительные запросы. Дополнительно были сформированы события аудита, переданные в сервис мониторинга и сохраненные в Elasticsearch.

Данный эксперимент подтверждает, что разработанный паттерн безопасности способен не только контролировать доступ, но и активно реагировать на потенциальные угрозы хищения данных.

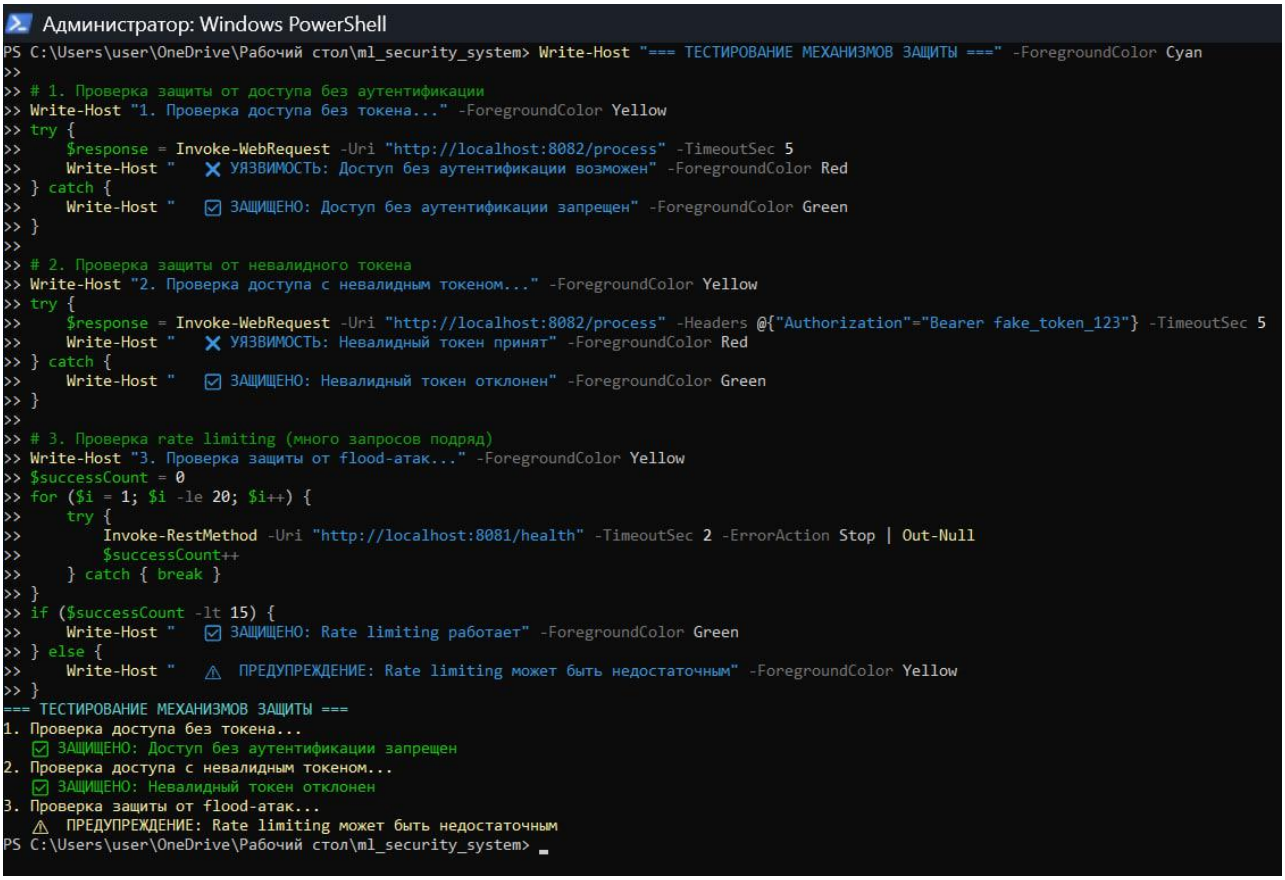


Рисунок 23 – Тестирование и обнаружение аномальной активности

В результате проведения тестирования микросервисов разработанной системы безопасности была подтверждена корректность развертывания всех компонентов в контейнеризированной среде. Использование Docker Compose позволило обеспечить изоляцию микросервисов, корректный порядок их запуска и устойчивое сетевое взаимодействие между компонентами системы.

Проверка состояния контейнеров в Docker Desktop показала, что все основные микросервисы — сервис аутентификации и авторизации, сервис обработки обучающих данных, сервис оценки безопасности, сервис мониторинга и аудита, а также сервис хранения обучающих данных — находятся в рабочем состоянии и успешно функционируют в рамках единой инфраструктуры. Это подтверждает корректность конфигурации Docker-образов и параметров окружения.

Автоматизированная проверка доступности микросервисов с использованием PowerShell-скриптов продемонстрировала стабильную работу health-эндпоинтов всех сервисов. Полученные результаты свидетельствуют о том, что каждый компонент системы корректно обрабатывает входящие запросы

и готов к выполнению своих функциональных обязанностей в рамках архитектурного паттерна безопасности.

Функциональное тестирование защитных механизмов показало, что реализованная ролевая модель доступа работает корректно и позволяет эффективно разграничивать права пользователей при обращении к обучающим данным. Попытки несанкционированного доступа, а также сценарии, имитирующие хищение данных и массовые операции выгрузки, были успешно выявлены и заблокированы сервисом безопасности. При этом соответствующие события корректно регистрировались в системе аудита.

Таким образом, результаты тестирования подтверждают, что разработанная микросервисная архитектура обеспечивает централизованный контроль доступа, предотвращает обход защитных механизмов и соответствует поставленным требованиям по защите обучающих данных в приложениях машинного обучения.

4.2 МОНИТОРИНГ МИКРОСЕРВИСОВ

После подтверждения корректности развертывания и базовой работоспособности микросервисов следующим этапом практического исследования является мониторинг состояния системы безопасности в процессе функционирования. Цель данного этапа заключается в наблюдении за ключевыми метриками работы микросервисов, а также в оценке стабильности и надежности разработанного паттерна в условиях длительной эксплуатации.

Мониторинг микросервисной архитектуры позволяет своевременно выявлять отклонения в работе компонентов, потенциальные проблемы с потреблением ресурсов и аномальные состояния, которые могут указывать на попытки нарушения безопасности или некорректную работу отдельных сервисов. В рамках данной работы мониторинг реализован с использованием системы сбора метрик Prometheus и средства визуализации Grafana.

Prometheus осуществляет периодический опрос всех микросервисов по специально настроенным endpoint’ам и собирает метрики, характеризующие состояние системы. Grafana, в свою очередь, используется для построения

наглядных дашбордов и графиков, позволяющих в реальном времени анализировать поведение микросервисов.

Одной из базовых, но критически важных метрик является показатель доступности сервисов (up). Данная метрика принимает значение 1 в случае корректной работы сервиса и значение 0 при его недоступности. Анализ этой метрики позволяет оперативно выявлять сбои, перезапуски контейнеров или ошибки конфигурации.

На рисунке 24 представлен график метрики up, отображающий состояние доступности всех микросервисов системы безопасности. В течение всего периода наблюдения значения метрики остаются равными 1 для каждого сервиса, что свидетельствует о стабильной работе контейнеров и отсутствии сбоев в функционировании компонентов системы.

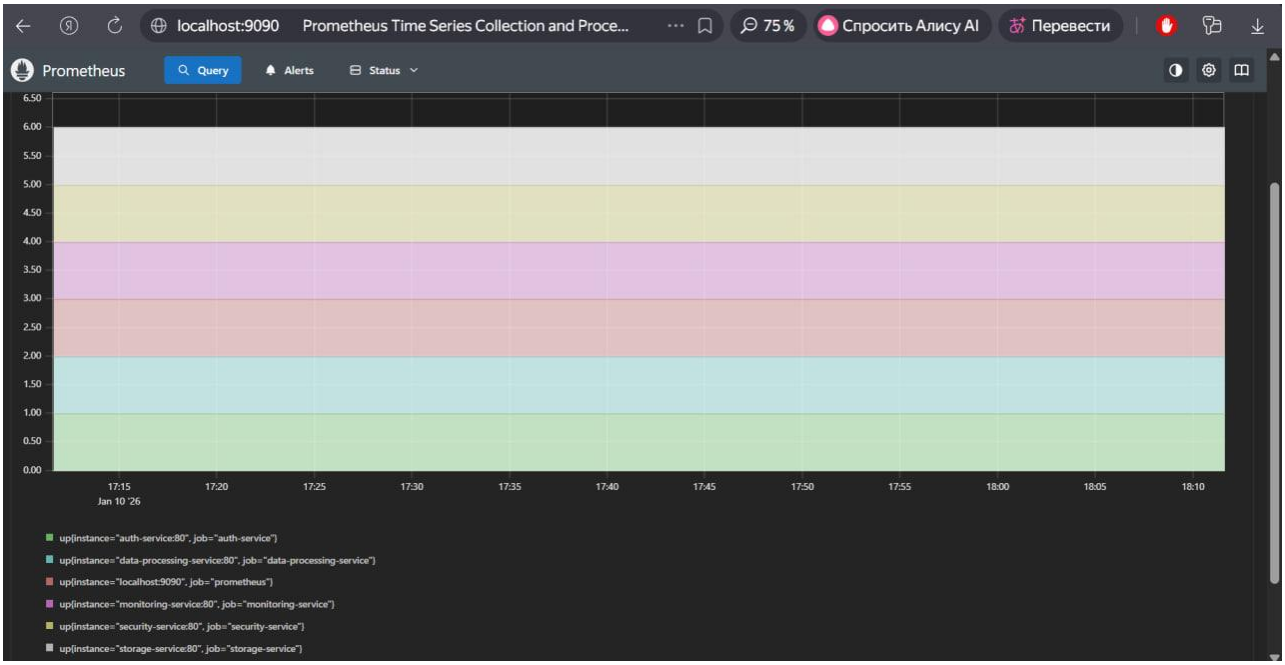


Рисунок 24 — Мониторинг доступности микросервисов в Prometheus

Следующим важным аспектом мониторинга является контроль использования файловых дескрипторов. Избыточное количество открытых дескрипторов может свидетельствовать об утечках ресурсов, некорректной работе сетевых соединений или ошибках при обработке запросов, что потенциально может привести к отказу в обслуживании.

На рисунке 25 представлен график количества открытых файловых дескрипторов для каждого микросервиса. Значения метрик находятся в

стабильном диапазоне от 10 до 30 дескрипторов на сервис, что соответствует нормальному режиму работы приложений. Отсутствие резких скачков подтверждает корректное управление ресурсами операционной системы и отсутствие аномального поведения сервисов.



Рисунок 25 — Мониторинг открытых файловых дескрипторов микросервисов

Использование оперативной памяти является одной из ключевых характеристик стабильности микросервисной архитектуры. Рост потребления памяти без последующего освобождения может указывать на утечки памяти или ошибки в логике обработки данных.

На рисунке 26 показана визуализация использования оперативной памяти (RSS — Resident Set Size) различными микросервисами системы. Видно, что сервисы мониторинга и обработки обучающих данных потребляют больший объем памяти по сравнению с сервисами аутентификации и безопасности, что обусловлено их функциональной нагрузкой. При этом значения остаются стабильными, без резких всплесков, что свидетельствует об отсутствии утечек памяти и корректной работе сервисов.

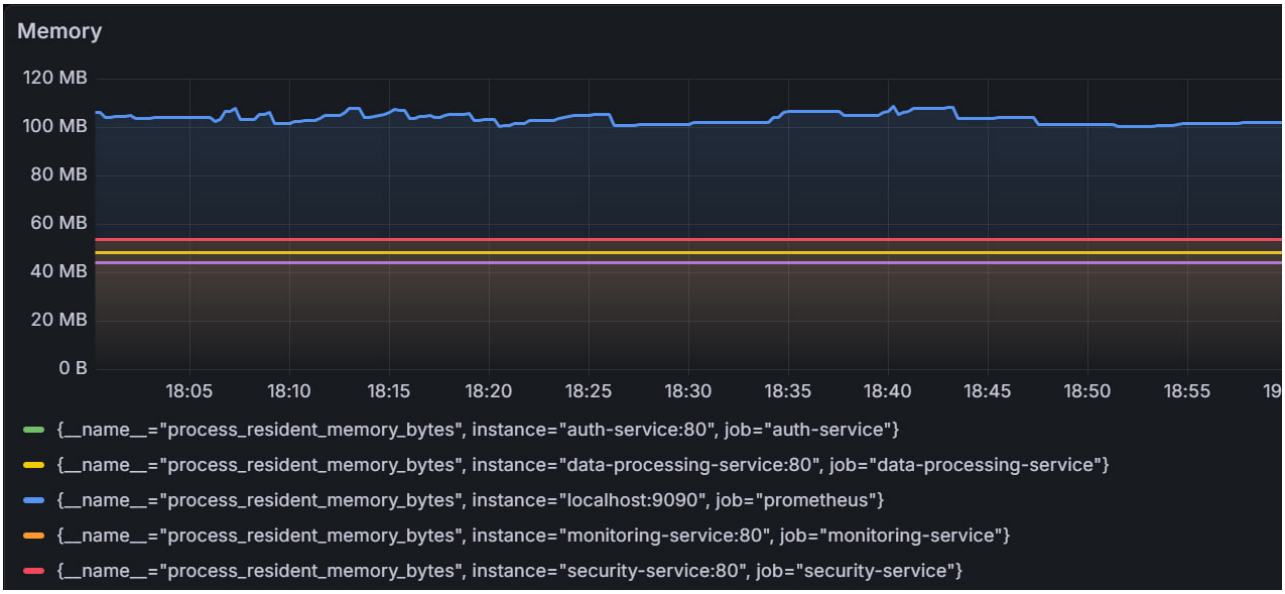


Рисунок 26 — Потребление оперативной памяти микросервисами

Для анализа сетевой активности и корректности обработки запросов используется метрика количества HTTP-запросов, обрабатываемых микросервисами. Данная метрика позволяет оценить интенсивность взаимодействия компонентов системы и корректность учета входящих обращений.

На рисунке 27 представлен график счетчиков HTTP-запросов. В условиях тестовой среды без активной пользовательской нагрузки значения остаются стабильными, что подтверждает корректную работу механизмов учета сетевой активности. Наличие отдельных линий для каждого микросервиса демонстрирует независимость компонентов и корректную изоляцию их функциональности.



Рисунок 27 — Счетчики HTTP-запросов микросервисов

Загрузка центрального процессора является важным показателем эффективности работы микросервисной архитектуры. Высокие значения CPU могут указывать на чрезмерную нагрузку, ошибки в реализации алгоритмов или необходимость масштабирования сервисов.

На рисунке 28 представлен график использования процессорного времени всеми микросервисами системы безопасности. Значения загрузки CPU находятся в диапазоне 0–5 %, что соответствует режиму ожидания и отсутствию интенсивных вычислительных операций. Более высокая активность сервисов мониторинга и безопасности объясняется выполнением фоновых операций анализа и обработки событий аудита.

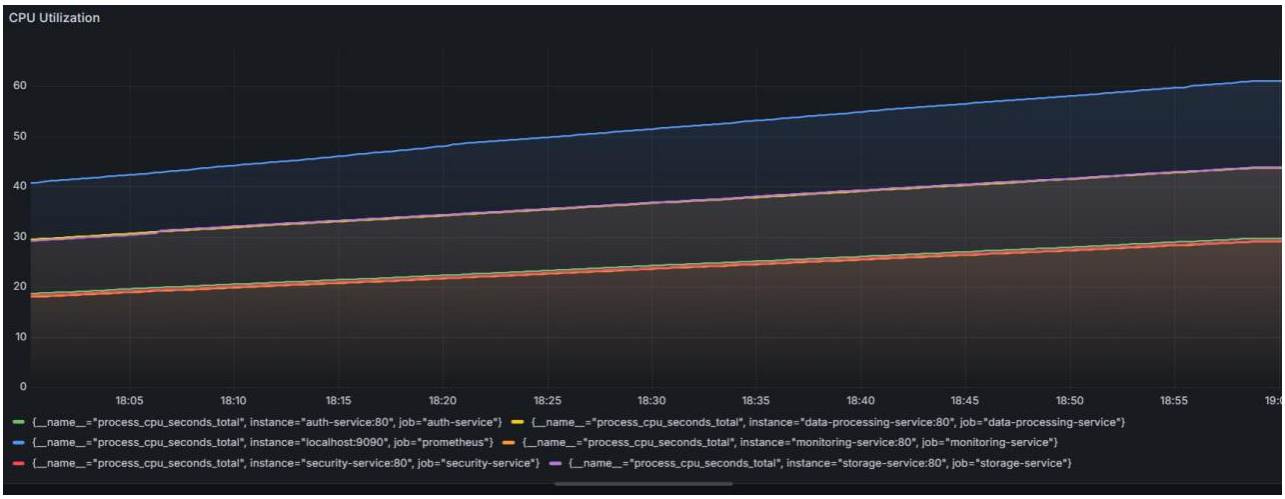


Рисунок 28 — Загрузка процессора микросервисами

Результаты мониторинга микросервисной системы безопасности подтвердили стабильность и устойчивость функционирования разработанного паттерна в условиях длительной эксплуатации. Использование системы сбора метрик Prometheus и средства визуализации Grafana позволило получить наглядное представление о состоянии всех компонентов системы в реальном времени.

Анализ метрики доступности сервисов (up) показал, что все микросервисы системы безопасности находились в рабочем состоянии на протяжении всего периода наблюдения. Отсутствие переходов метрики в неактивное состояние свидетельствует о надежности контейнеризированной инфраструктуры и корректной работе механизмов оркестрации.

Контроль использования файловых дескрипторов и оперативной памяти показал, что микросервисы потребляют системные ресурсы в пределах допустимых значений. Отсутствие резких скачков и аномальных значений указывает на корректную реализацию логики обработки запросов и отсутствие утечек ресурсов, которые могли бы привести к деградации производительности или отказу в обслуживании.

Анализ сетевой активности и счетчиков HTTP-запросов подтвердил корректность взаимодействия микросервисов между собой и точность механизмов учета входящих запросов. Раздельная регистрация сетевой активности для каждого сервиса обеспечивает прозрачность работы системы и позволяет выявлять потенциальные аномалии в поведении отдельных компонентов.

Мониторинг загрузки процессора показал, что система безопасности функционирует с минимальной нагрузкой на вычислительные ресурсы, что подтверждает эффективность реализованных алгоритмов и возможность масштабирования системы при росте нагрузки. Более высокая активность сервисов мониторинга и безопасности объясняется выполнением фоновых операций анализа и аудита, что является ожидаемым и допустимым поведением.

В целом проведенный мониторинг подтверждает, что разработанный паттерн безопасности обеспечивает устойчивую, предсказуемую и контролируемую работу микросервисной системы. Наличие централизованного мониторинга и визуализации метрик позволяет оперативно реагировать на потенциальные инциденты и обеспечивает высокий уровень надежности защиты обучающих данных.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был разработан и экспериментально исследован архитектурный паттерн обеспечения безопасности приложения машинного обучения при угрозе хищения обучающих данных. Актуальность выбранной темы обусловлена высокой ценностью обучающих датасетов и ростом числа угроз, направленных на их компрометацию в распределённых и микросервисных системах.

В первой главе работы был проведён анализ предметной области, в рамках которого обучающие данные рассмотрены как специфический объект защиты. С использованием методических рекомендаций ФСТЭК России были выделены и формализованы основные сценарии угроз хищения обучающих данных, включая несанкционированный доступ через уязвимые сервисы и компрометацию учётных данных привилегированных пользователей. На основе данного анализа сформулированы требования к механизмам защиты и обоснована необходимость централизованного контроля доступа и мониторинга операций.

Во второй главе разработан архитектурный паттерн обеспечения безопасности приложения машинного обучения, основанный на микросервисном подходе. Предложенная архитектура предусматривает разделение ответственности между специализированными микросервисами аутентификации, обработки данных, оценки безопасности, мониторинга и хранения обучающих данных. Ключевой особенностью паттерна является исключение прямого доступа сервисов обработки к хранилищу данных и обязательное прохождение всех запросов через сервис безопасности. Для формализации логики взаимодействия компонентов были разработаны UML-диаграммы последовательности, состояний и деятельности.

В третьей главе выполнена практическая реализация разработанного паттерна в виде программного прототипа. Реализованы микросервисы на языке Python, обеспечивающие контроль доступа, анализ угроз, аудит событий безопасности и защищённое хранение обучающих данных. Проведена контейнеризация компонентов с использованием Docker и подготовлены

конфигурации для развертывания в среде Docker Compose и Kubernetes. Такой подход позволил обеспечить изоляцию сервисов, масштабируемость и отказоустойчивость системы.

В четвёртой главе выполнено комплексное тестирование и мониторинг разработанной микросервисной архитектуры. Экспериментальная часть подтвердила корректность развертывания всех компонентов системы и их стабильную работу в контейнеризированной среде. Автоматизированные проверки доступности микросервисов, а также сценарии имитации аномальной активности показали, что реализованные механизмы безопасности эффективно выявляют и блокируют попытки несанкционированного доступа к обучающим данным. Использование Prometheus и Grafana позволило провести мониторинг ключевых метрик производительности и устойчивости системы, подтвердив отсутствие утечек ресурсов и стабильное функционирование сервисов.

Таким образом, поставленная цель курсовой работы была достигнута. Разработанный архитектурный паттерн доказал свою работоспособность и эффективность в условиях, приближенных к реальной эксплуатации. Полученные результаты подтверждают возможность практического применения предложенного решения для защиты обучающих данных в распределённых ML-приложениях. Разработанный прототип может быть использован как основа для дальнейшего развития систем безопасности, а также для интеграции в современные DevOps и MLOps-процессы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. A Security Analysis Model for IoT-ecosystem Using Machine Learning- (ML) Approach / P. K. N S, M. P. Kantipudi, P. N [et al.] // Recent Advances in Computer Science and Communications. – 2024. – Vol. 17. – DOI 10.2174/0126662558286885240223093414. – EDN JYVOIJ. (дата обращения: 09.01.2026)
2. Docker Compose Documentation [Электронный ресурс]: сайт. – Режим доступа: https://docs.docker.com/compose/ (дата обращения: 09.01.2026)
3. Rethinking security: the resilience of shallow ML models / R. Teixeira, M. Antunes, J. P. Barraca [et al.] // International Journal of Data Science and Analytics. – 2024. – DOI 10.1007/s41060-024-00655-1. – EDN ETQGVY. (дата обращения: 09.01.2026)
4. Аль-Свейти, М. А. М. Федеративное обучение с сохранением конфиденциальности: баланс между точностью и защитой данных в распределенном машинном обучении / М. А. М. Аль-Свейти, З. В. Ким, Д. В. Маршев // Информационные технологии и телекоммуникации. – 2025. – Т. 13, № 2. – С. 52-68. – DOI 10.31854/2307-1303-2025-13-2-52-68. – EDN YHQXCK. (дата обращения: 09.01.2026)
5. Аскаров, Т. Е. Архитектура docker и технологии оркестровки: docker swarm и kubernetes / Т. Е. Аскаров // Научный Лидер. – 2024. – № 20(170). – С. 9-12. – EDN JXOSVI. (дата обращения: 09.01.2026)
6. База данных угроз ФСТЭК России // URL: https://bdu.fstec.ru/threat. (дата обращения: 09.01.2026)
7. Балута, В. И. Формирование синтетических данных при подготовке обучающих множеств для систем предупреждения ЧС / В. И. Балута // Препринты ИПМ им. М.В. Келдыша. – 2022. – № 60. – С. 1-28. – DOI 10.20948/prepr-2022-60. – EDN UWPJZC. (дата обращения: 09.01.2026)
8. Буранова, М. А. Использование методов машинного обучения в задачах кластеризации в области информационной безопасности / М. А. Буранова, В. Г. Карташевский // Актуальные проблемы информатики, радиотехники и связи : Материалы XXXI Российской научно-технической конференции,

Самара, 01–02 февраля 2024 года. – Самара: Поволжский государственный университет телекоммуникаций и информатики, 2024. – С. 86-87. – EDN AYCMWI. (дата обращения: 09.01.2026)

9. Доргушаова, А. К. Основные аспекты процесса установления уровня защищенности в информационных системах персональных данных / А. К. Доргушаова, Д. И. Шередько // Актуальные вопросы науки и образования. – 2023. – № 1. – С. 57-61. – EDN QDDCLM. (дата обращения: 09.01.2026)
10. Корнеев Н.В. Методы и алгоритмы проектирования отечественной облачной платформы промышленного интернета вещей для интеллектуального управления комплексной безопасностью промышленных объектов нефтегазового комплекса / Корнеев Н.В. — М. ИЦ РГУ нефти и газа (НИУ) имени И.М. Губкина, 2024. — 103 с., 6,45 п. л. 978-5-91961-562-0 (Библиотека РГУ нефти и газа) [Электронный ресурс] URL: https://elib.gubkin.ru/#/catalog/item/3753 (дата обращения: 09.01.2026)
11. Корнеев Н.В. Языки программирования / Корнеев Н.В. — М. учеб. пособие, 2024. — 240 с., 15 п. л. 978-5-91961-529-3 (Библиотека РГУ нефти и газа) [Электронный ресурс] URL: https://elib.gubkin.ru/#/catalog/item/3559 (дата обращения: 09.01.2026)
12. Корнеев, Н. В. Паттерн для обеспечения безопасности веб приложения при угрозе XSS атак в облачной инфраструктуре / Н. В. Корнеев, Д. С. Лазорин // Вопросы кибербезопасности. – 2024. – № 6(64). – С. 76-84. – DOI 10.21681/2311-3456-2024-6-76-84. – EDN DIVBQJ. (дата обращения: 09.01.2026)
13. Корнеев, Н. В. Паттерн для обеспечения безопасности информационной инфраструктурыпри миграции образов виртуальных машин / Н. В. Корнеев, А. Б. Дикий // Вопросы кибербезопасности. – 2025. – № 2(66). – С. 29-40. – DOI 10.21681/2311-3456-2025-2-29-40. – EDN NLUUOR. (дата обращения: 09.01.2026)
14. Корнеев, Н. В. Паттерн для обеспечения безопасности приложения при угрозе модификации модели машинного обучения / Н. В. Корнеев, Е. С.

Котрини // Вопросы кибербезопасности. – 2025. – № 1(65). – С. 117-127. – DOI 10.21681/2311-3456-2025-1-117-127. – EDN RNHFEC. (дата обращения: 09.01.2026)

15. Корнеев, Н. В. Паттерн для обеспечения безопасностивеб приложений при угрозе неконтролируемого роста числа зарезервированных ресурсов / Н. В. Корнеев, А. Е. Трубачева-Гудович // Вопросы 59 кибербезопасности. – 2025. – № 4(68). – С. 35-45. – DOI 10.21681/2311-3456 2025-4-35-45. – EDN RWPQPB. (дата обращения: 09.01.2026)
16. Корсакова, С. К. Прогнозирование временного ряда в Python с помощью метода Хольта-Уинтерса / С. К. Корсакова, Н. В. Русина, В. М. Бондарик // BIG DATA и анализ высокого уровня : сборник научных статей X Международной научно-практической конференции, Минск, 13 марта 2024 года. – Минск: Белорусский государственный университет информатики и радиоэлектроники, 2024. – С. 438-443. – EDN DCXSAB. (дата обращения: 09.01.2026)
17. Мурылева, А. А. Защита моделей машинного обучения от извлечения обучающего набора данных / А. А. Мурылева, М. О. Калинин, Д. С. Лаврова // Проблемы информационной безопасности. Компьютерные системы. – 2024. – № 1(58). – С. 142-152. – DOI 10.48612/jisp/en1z-upfz-u3p7. – EDN AVORQP. (дата обращения: 09.01.2026)
18. Паттерны микросервисной архитектуры: многопрофильный обзор литературы / Х. А. Вальдивия, А. Лора-Гонсалес, К. Лимон [и др.] // Труды Института системного программирования РАН. – 2021. – Т. 33, № 1. – С. 81-96. – DOI 10.15514/ISPRAS-2021-33(1)-6. – EDN MHZQIC. (дата обращения: 09.01.2026)
19. Плешакова, Е. С. Анализ и оценка угроз безопасности и конфиденциальности в сетях 5G с использованием машинного обучения / Е. С. Плешакова, Н. Э. Карпенко, С. М. Гомбоев // Вопросы обеспечения безопасности в киберпространстве : материалы Всероссийской научно-технической конференции, Махачкала, 16 декабря 2022 года. – Махачкала: ДГТУ, 2022. – С. 310-315. – EDN EDBHDB. (дата обращения: 09.01.2026)

20. Положение о курсовом проектировании в РГУ нефти и газа (НИУ) имени И.М. Губкина (приказ от 11.11.2021 № 398) (дата обращения: 09.01.2026)
21. Разработка архитектуры машинного обучения с использованием гомоморфного шифрования для обеспечения конфиденциальности данных / В. А. Частикова, С. А. Жерлицын, А. Н. Пешков, А. С. Карапетян // Электронный сетевой политематический журнал "Научные труды КубГТУ". – 2022. – № 2. – С. 135-147. – EDN KTRKHW. (дата обращения: 09.01.2026)
22. Рудницкая, Е. А. Конфиденциальность моделей машинного обучения / Е. А. Рудницкая, М. А. Полтавцева // Проблемы информационной безопасности. Компьютерные системы. – 2023. – № S2(55). – С. 108-119. – DOI 10.48612/jisp/e9tz-1ge5-ke25. – EDN CBCKBQ. (дата обращения: 09.01.2026)
23. Свидетельство о государственной регистрации базы данных № 2021622152 Российская Федерация. Обучающая база данных "Учебно-методические материалы и тестовые вопросы по программе дополнительного профессионального образования "Повышение квалификации специалистов (экспертов-аудиторов), осуществляющих работы по проверке работоспособности систем и элементов противопожарной защиты" : № 2021622071 : заявл. 11.10.2021 : опубл. 15.10.2021 / О. Д. Ратникова, Н. В. Перегудова, Д. Н. Куркин [и др.] ; заявитель Федеральное государственное бюджетное учреждение «Всероссийский ордена «Знак Почета» научно-исследовательский институт противопожарной обороны Министерства Российской Федерации по делам гражданской обороны, чрезвычайным ситуациям и ликвидации последствий стихийных бедствий». – EDN IDTZKQ. (дата обращения: 09.01.2026)
24. Свидетельство о государственной регистрации программы для ЭВМ № 2024663461 Российская Федерация. Программная модель оценивания применяемых механизмов обеспечения конфиденциальности обучающих наборов данных и точности дифференциальных моделей машинного

обучения : № 2024662290 : заявл. 01.06.2024 : опубл. 06.06.2024 / Д. Ю. Казьмин. – EDN QUXAGP. (дата обращения: 09.01.2026)

25. Свидетельство о государственной регистрации программы для ЭВМ № 2024691847 Российская Федерация. Программное обеспечение для защиты от копирования обученных моделей машинного обучения и защиты от извлечения обучающих данных из обученных моделей : заявл. 12.12.2024 : опубл. 24.12.2024 / К. В. Архипенко, Д. Д. Прилепская, А. В. Пивоваров [и др.] ; заявитель Федеральное государственное бюджетное учреждение науки Институт системного программирования им. В.П. Иванникова Российской академии наук. – EDN KOVEPQ. (дата обращения: 09.01.2026)
26. Сейдаметов, С. В. Миграция от монолитов к микросервисам и их интеграция в облачные системы / С. В. Сейдаметов, С. Сейдаметова, Э. А. Кадырова // Ученые записки Крымского инженерно-педагогического университета. – 2024. – № 2(84). – С. 204-209. – EDN GUYMTO. (дата обращения: 09.01.2026)
27. Технология корпоративной защиты персональных данных и конфиденциальной информации / О. Ю. Башарина, Е. В. Буценко, Е. О. Похомчикова, И. С. Шильникова // Современные наукоемкие технологии. – 2024. – № 2. – С. 8-14. – DOI 10.17513/snt.39924. – EDN JMCXPA. (дата обращения: 09.01.2026)
28. Топольский, Н. Г. Методы, модели и алгоритмы в системах безопасности : машинное обучение, робототехника, страхование, риски, контроль / Н. Г. Топольский, В. Я. Вилисов. – Москва : ООО "Издательский Центр РИОР", 2021. – 475 с. – ISBN 978-5-369-02072-2. – DOI 10.29039/02072-2. – EDN XJPGSH. (дата обращения: 09.01.2026)
29. Цехановский, В. В. Проектирование информационных систем: архитектуры и платформы : Учебное пособие / В. В. Цехановский, А. И. Водяхо. – Москва : Ай Пи Ар Медиа, 2023. – 240 с. – ISBN 978-5-4497-1786-3. – EDN DVKNFP. (дата обращения: 09.01.2026)

30. Ященко, В. А. Разработка комплексной методики повышения безопасности обработки конфиденциальной информации в производстве / В. А. Ященко // Материалы 79-й студенческой научной конференции : сборник статей, Брянск, 18–22 марта 2024 года. – Брянск: Брянский государственный технический университет, 2024. – С. 729-731. – EDN EUYZEQ. (дата обращения: 09.01.2026)

ПРИЛОЖЕНИЕ А

1. Код микросервиса аутентификации и авторизации (Authentication and Authorization Service)

```
from fastapi import FastAPI, Request
from prometheus_client import Counter, Histogram, generate_latest
from fastapi.responses import Response
import uvicorn
import time

# ===== METRICS WITH LABELS =====
HTTP_REQUESTS_TOTAL = Counter(
    'http_requests_total',
    'Total HTTP requests',
    ['method', 'endpoint', 'status_code', 'service']
)

HTTP_REQUEST_DURATION = Histogram(
    'http_request_duration_seconds',
    'HTTP request duration in seconds',
    ['method', 'endpoint', 'service'],
    buckets=[0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
)

AUTH_REQUESTS = Counter(
    'auth_requests_total',
    'Total requests for auth service',
    ['type']
)

# ===== CREATE APP =====
app = FastAPI()

# ===== METRICS MIDDLEWARE =====
@app.middleware("http")
async def metrics_middleware(request: Request, call_next):
    print(f"MIDDLEWARE: {request.method} {request.url.path}")
    start_time = time.time()
    endpoint = request.url.path

    try:
        response = await call_next(request)

        print(f"☑ Request completed: {request.method} {endpoint}
-> {response.status_code}")

        HTTP_REQUESTS_TOTAL.labels(
            method=request.method,
            endpoint=endpoint,
            status_code=response.status_code,
            service="auth-service"
        ).inc()
```

```
        print(f"Metric incremented for {endpoint}")

    except Exception as e:
        print(f"❌ Request error: {e}")
        HTTP_REQUESTS_TOTAL.labels(
            method=request.method,
            endpoint=endpoint,
            status_code=500,
            service="auth-service"
        ).inc()
        raise e

    finally:
        duration = time.time() - start_time
        HTTP_REQUEST_DURATION.labels(
            method=request.method,
            endpoint=endpoint,
            service="auth-service"
        ).observe(duration)
        print(f"Duration: {duration:.3f}s")

    return response

# ===== ENDPOINTS (ALL ASYNC!) =====
@app.get("/health")
async def health():  # ← async добавлено!
    print("Health endpoint called")
    AUTH_REQUESTS.labels(type="health").inc()
    return {"status": "healthy", "service": "Auth Service"}

@app.get("/metrics")
async def metrics():  # ← async добавлено!
    print("Metrics endpoint called")
    AUTH_REQUESTS.labels(type="metrics").inc()
    return Response(generate_latest(), media_type="text/plain")

@app.get("/")
async def root():  # ← async добавлено!
    print("Root endpoint called")
    AUTH_REQUESTS.labels(type="root").inc()
    return {"message": "Auth Service is running"}

@app.get("/users")
async def get_users():  # ← async добавлено!
    print("Users endpoint called")
    return {"users": ["user1", "user2"]}

@app.post("/login")
async def login():  # ← async добавлено!
    print("Login endpoint called")
    return {"token": "fake-token"}

# ===== RUN =====
if __name__ == "__main__":
```

```
    print("Starting Auth Service with metrics middleware...")
    uvicorn.run(app, host="0.0.0.0", port=80, log_level="info")
```

2. Код микросервиса обработки обучающих данных (Training Data Processing Service)

```
from fastapi import FastAPI, Request
from prometheus_client import Counter, Histogram, generate_latest
from fastapi.responses import Response
import uvicorn
import time

# ===== METRICS WITH LABELS =====
HTTP_REQUESTS_TOTAL = Counter(
    'http_requests_total',
    'Total HTTP requests',
    ['method', 'endpoint', 'status_code', 'service']
)

HTTP_REQUEST_DURATION = Histogram(
    'http_request_duration_seconds',
    'HTTP request duration in seconds',
    ['method', 'endpoint', 'service'],
    buckets=[0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
)

DATA_PROCESSING_REQUESTS = Counter(
    'data_processing_requests_total',
    'Total requests for data processing service',
    ['type']
)

# ===== CREATE APP =====
app = FastAPI()

# ===== METRICS MIDDLEWARE =====
@app.middleware("http")
async def metrics_middleware(request: Request, call_next):
    start_time = time.time()
    endpoint = request.url.path

    try:
        response = await call_next(request)

        HTTP_REQUESTS_TOTAL.labels(
            method=request.method,
            endpoint=endpoint,
            status_code=response.status_code,
            service="data-processing-service"
        ).inc()

    except Exception as e:
        HTTP_REQUESTS_TOTAL.labels(
            method=request.method,
            endpoint=endpoint,
```

```
            status_code=500,
            service="data-processing-service"
        ).inc()
        raise e

    finally:
        duration = time.time() - start_time
        HTTP_REQUEST_DURATION.labels(
            method=request.method,
            endpoint=endpoint,
            service="data-processing-service"
        ).observe(duration)

    return response

# ===== ENDPOINTS =====
@app.get("/health")
def health():
    DATA_PROCESSING_REQUESTS.labels(type="health").inc()
    return {"status": "healthy", "service": "Data Processing
Service"}

@app.get("/metrics")
def metrics():
    DATA_PROCESSING_REQUESTS.labels(type="metrics").inc()
    return Response(generate_latest(), media_type="text/plain")

@app.get("/")
def root():
    DATA_PROCESSING_REQUESTS.labels(type="root").inc()
    return {"message": "Data Processing Service is running"}

@app.post("/process")
def process_data():
    return {"processed": True, "items": 100}

# ===== RUN =====
if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=80)
```

3. Код микросервиса безопасности (Security Evaluation Service)

```
from fastapi import FastAPI, Request
from prometheus_client import Counter, Histogram, generate_latest
from fastapi.responses import Response
import uvicorn
import time

# ===== METRICS WITH LABELS =====
HTTP_REQUESTS_TOTAL = Counter(
    'http_requests_total',
    'Total HTTP requests',
    ['method', 'endpoint', 'status_code', 'service']
)
```

```
HTTP_REQUEST_DURATION = Histogram(
    'http_request_duration_seconds',
    'HTTP request duration in seconds',
    ['method', 'endpoint', 'service'],
    buckets=[0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
)

SECURITY_REQUESTS = Counter(
    'security_requests_total',
    'Total requests for security service',
    ['type']
)

# ===== CREATE APP =====
app = FastAPI()

# ===== METRICS MIDDLEWARE =====
@app.middleware("http")
async def metrics_middleware(request: Request, call_next):
    start_time = time.time()
    endpoint = request.url.path

    try:
        response = await call_next(request)

        HTTP_REQUESTS_TOTAL.labels(
            method=request.method,
            endpoint=endpoint,
            status_code=response.status_code,
            service="security-service"
        ).inc()

    except Exception as e:
        HTTP_REQUESTS_TOTAL.labels(
            method=request.method,
            endpoint=endpoint,
            status_code=500,
            service="security-service"
        ).inc()
        raise e

    finally:
        duration = time.time() - start_time
        HTTP_REQUEST_DURATION.labels(
            method=request.method,
            endpoint=endpoint,
            service="security-service"
        ).observe(duration)

    return response

# ===== ENDPOINTS =====
@app.get("/health")
def health():
```

```
    SECURITY_REQUESTS.labels(type="health").inc()
    return {"status": "healthy", "service": "Security Service"}

@app.get("/metrics")
def metrics():
    SECURITY_REQUESTS.labels(type="metrics").inc()
    return Response(generate_latest(), media_type="text/plain")

@app.get("/")
def root():
    SECURITY_REQUESTS.labels(type="root").inc()
    return {"message": "Security Service is running"}

@app.get("/scan")
def security_scan():
    return {"threats": 0, "status": "clean"}

# ===== RUN =====
if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=80)
```

4. Код микросервиса мониторинга и аудита безопасности (Security Monitoring and Audit Service)

```
from fastapi import FastAPI, Request
from prometheus_client import Counter, Histogram, generate_latest
from fastapi.responses import Response
import uvicorn
import time

# ===== METRICS WITH LABELS =====
HTTP_REQUESTS_TOTAL = Counter(
    'http_requests_total',
    'Total HTTP requests',
    ['method', 'endpoint', 'status_code', 'service']
)

HTTP_REQUEST_DURATION = Histogram(
    'http_request_duration_seconds',
    'HTTP request duration in seconds',
    ['method', 'endpoint', 'service'],
    buckets=[0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
)

MONITORING_REQUESTS = Counter(
    'monitoring_requests_total',
    'Total requests for monitoring service',
    ['type']
)

# ===== CREATE APP =====
app = FastAPI()

# ===== METRICS MIDDLEWARE =====
@app.middleware("http")
```

```
async def metrics_middleware(request: Request, call_next):
    start_time = time.time()
    endpoint = request.url.path

    try:
        response = await call_next(request)

        HTTP_REQUESTS_TOTAL.labels(
            method=request.method,
            endpoint=endpoint,
            status_code=response.status_code,
            service="monitoring-service"
        ).inc()

    except Exception as e:
        HTTP_REQUESTS_TOTAL.labels(
            method=request.method,
            endpoint=endpoint,
            status_code=500,
            service="monitoring-service"
        ).inc()
        raise e

    finally:
        duration = time.time() - start_time
        HTTP_REQUEST_DURATION.labels(
            method=request.method,
            endpoint=endpoint,
            service="monitoring-service"
        ).observe(duration)

    return response

# ===== ENDPOINTS =====
@app.get("/health")
def health():
    MONITORING_REQUESTS.labels(type="health").inc()
    return {"status": "healthy", "service": "Monitoring Service"}

@app.get("/metrics")
def metrics():
    MONITORING_REQUESTS.labels(type="metrics").inc()
    return Response(generate_latest(), media_type="text/plain")

@app.get("/")
def root():
    MONITORING_REQUESTS.labels(type="root").inc()
    return {"message": "Monitoring Service is running"}

@app.get("/status")
def system_status():
    return {"services": 5, "healthy": 5}

# ===== RUN =====
```

```
if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=80)
```

5. Код микросервиса хранения обучающих данных (Training Data Storage Service)

```
from fastapi import FastAPI, Request
from prometheus_client import Counter, Histogram, generate_latest
from fastapi.responses import Response
import uvicorn
import time

# ===== METRICS WITH LABELS =====
HTTP_REQUESTS_TOTAL = Counter(
    'http_requests_total',
    'Total HTTP requests',
    ['method', 'endpoint', 'status_code', 'service']
)

HTTP_REQUEST_DURATION = Histogram(
    'http_request_duration_seconds',
    'HTTP request duration in seconds',
    ['method', 'endpoint', 'service'],
    buckets=[0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
)

STORAGE_REQUESTS = Counter(
    'storage_requests_total',
    'Total requests for storage service',
    ['type']
)

# ===== CREATE APP =====
app = FastAPI()

# ===== METRICS MIDDLEWARE =====
@app.middleware("http")
async def metrics_middleware(request: Request, call_next):
    start_time = time.time()
    endpoint = request.url.path

    try:
        response = await call_next(request)

        HTTP_REQUESTS_TOTAL.labels(
            method=request.method,
            endpoint=endpoint,
            status_code=response.status_code,
            service="storage-service"
        ).inc()

    except Exception as e:
        HTTP_REQUESTS_TOTAL.labels(
            method=request.method,
            endpoint=endpoint,
```

```
            status_code=500,
            service="storage-service"
        ).inc()
        raise e

    finally:
        duration = time.time() - start_time
        HTTP_REQUEST_DURATION.labels(
            method=request.method,
            endpoint=endpoint,
            service="storage-service"
        ).observe(duration)

    return response

# ===== ENDPOINTS =====
@app.get("/health")
def health():
    STORAGE_REQUESTS.labels(type="health").inc()
    return {"status": "healthy", "service": "Storage Service"}

@app.get("/metrics")
def metrics():
    STORAGE_REQUESTS.labels(type="metrics").inc()
    return Response(generate_latest(), media_type="text/plain")

@app.get("/")
def root():
    STORAGE_REQUESTS.labels(type="root").inc()
    return {"message": "Storage Service is running"}

@app.get("/files")
def list_files():
    return {"files": ["file1.txt", "file2.csv"]}

# ===== RUN =====
if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=80)
```

6. docker-compose.yml

```
version: '3.8'

services:
  # 1. Микросервис аутентификации и авторизации
  auth-service:
    build: ./auth-service
    ports:
      - '8081:80'
    environment:
      DATABASE_URL:
'postgresql://user:password@postgres:5432/security_db'
      REDIS_URL: 'redis://redis:6379'
      JWT_SECRET: 'your_super_secret_jwt_key'
    networks:
```

```
      - security-net
    depends_on:
      - postgres
      - redis
    healthcheck:
      test: ['CMD', 'curl', '-f', 'http://localhost:80/health']
      interval: 30s
      timeout: 10s
      retries: 3

  # 2. Микросервис обработки обучающих данных
  data-processing-service:
    build: ./data-processing-service
    ports:
      - '8082:80'
    environment:
      DATABASE_URL:
'postgresql://user:password@postgres:5432/security_db'
      SECURITY_SERVICE_URL: 'http://security-service:80'
    networks:
      - security-net
    depends_on:
      - postgres
      - security-service
    healthcheck:
      test: ['CMD', 'curl', '-f', 'http://localhost:80/health']
      interval: 30s
      timeout: 10s
      retries: 3

  # 3. Микросервис безопасности
  security-service:
    build: ./security-service
    ports:
      - '8083:80'
    environment:
      DATABASE_URL:
'postgresql://user:password@postgres:5432/security_db'
      REDIS_URL: 'redis://redis:6379'
    networks:
      - security-net
    depends_on:
      - postgres
      - redis
    healthcheck:
      test: ['CMD', 'curl', '-f', 'http://localhost:80/health']
      interval: 30s
      timeout: 10s
      retries: 3

  # 4. Микросервис мониторинга и аудита безопасности
  monitoring-service:
    build: ./monitoring-service
    ports:
```

```
      - '8084:80'
    environment:
      DATABASE_URL:
'postgresql://user:password@postgres:5432/security_db'
      ELASTICSEARCH_URL: 'http://elasticsearch:9200'
    networks:
      - security-net
    depends_on:
      - postgres
      - elasticsearch
    healthcheck:
      test: ['CMD', 'curl', '-f', 'http://localhost:80/health']
      interval: 30s
      timeout: 10s
      retries: 3

  # 5. Микросервис хранения обучающих данных
  storage-service:
    build: ./storage-service
    ports:
      - '8085:80'
    environment:
      DATABASE_URL:
'postgresql://user:password@postgres:5432/security_db'
      MINIO_ENDPOINT: 'minio:9000'
      MINIO_ACCESS_KEY: 'minioadmin'
      MINIO_SECRET_KEY: 'minioadmin'
    networks:
      - security-net
    depends_on:
      - postgres
      - minio
    healthcheck:
      test: ['CMD', 'curl', '-f', 'http://localhost:80/health']
      interval: 30s
      timeout: 10s
      retries: 3

  # Вспомогательные сервисы
  postgres:
    image: postgres:13-alpine
    environment:
      POSTGRES_DB: 'security_db'
      POSTGRES_USER: 'user'
      POSTGRES_PASSWORD: 'password'
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - security-net
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U user -d security_db']
      interval: 30s
      timeout: 10s
      retries: 3
```

```
  redis:
    image: redis:7-alpine
    networks:
      - security-net
    healthcheck:
      test: ['CMD', 'redis-cli', 'ping']
      interval: 30s
      timeout: 10s
      retries: 3

  elasticsearch:
    image: elasticsearch:7.17.0
    environment:
      discovery.type: single-node
      ES_JAVA_OPTS: '-Xms512m -Xmx512m'
    volumes:
      - es_data:/usr/share/elasticsearch/data
    networks:
      - security-net
    healthcheck:
      test: ['CMD', 'curl', '-f',
'http://localhost:9200/_cluster/health']
      interval: 30s
      timeout: 10s
      retries: 3

  minio:
    image: minio/minio:latest
    command: server /data --console-address ':9001'
    environment:
      MINIO_ROOT_USER: 'minioadmin'
      MINIO_ROOT_PASSWORD: 'minioadmin'
    volumes:
      - minio_data:/data
    networks:
      - security-net
    healthcheck:
      test: ['CMD', 'curl', '-f',
'http://localhost:9000/minio/health/live']
      interval: 30s
      timeout: 10s
      retries: 3

  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus_data:/prometheus
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'
      - '--
web.console.libraries=/etc/prometheus/console_libraries'
```

```
      - '--web.console.templates=/etc/prometheus/consoles'
      - '--storage.tsdb.retention.time=200h'
      - '--web.enable-lifecycle'
    ports:
      - '9090:9090'
    networks:
      - security-net
    depends_on:
      - auth-service
      - data-processing-service
      - security-service
      - monitoring-service
      - storage-service

  grafana:
    image: grafana/grafana:latest
    environment:
      GF_SECURITY_ADMIN_PASSWORD: 'admin'
    volumes:
      - grafana_data:/var/lib/grafana
    ports:
      - '3000:3000'
    networks:
      - security-net
    depends_on:
      - prometheus

volumes:
  postgres_data:
  es_data:
  minio_data:
  prometheus_data:
  grafana_data:

networks:
  security-net:
    driver: bridge
```

7. prometheus.yml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'auth-service'
    static_configs:
```

```
      - targets: ['auth-service:80']
    metrics_path: '/metrics'
    scrape_interval: 10s

  - job_name: 'data-processing-service'
    static_configs:
      - targets: ['data-processing-service:80']
    metrics_path: '/metrics'
    scrape_interval: 10s

  - job_name: 'security-service'
    static_configs:
      - targets: ['security-service:80']
    metrics_path: '/metrics'
    scrape_interval: 10s

  - job_name: 'monitoring-service'
    static_configs:
      - targets: ['monitoring-service:80']
    metrics_path: '/metrics'
    scrape_interval: 10s

  - job_name: 'storage-service'
    static_configs:
      - targets: ['storage-service:80']
    metrics_path: '/metrics'
    scrape_interval: 10s
```

8. base_requirements.txt

```
# Base dependencies for ML Security System microservices
fastapi==0.104.1
uvicorn[standard]==0.24.0
prometheus-client==0.19.0
pydantic==2.5.0
python-multipart==0.0.6
httpx==0.25.1
```

9. metrics_middleware.py

```
"""
Prometheus Metrics Middleware для FastAPI приложений
Добавляет сбор HTTP метрик: количество запросов, время выполнения
"""

from prometheus_client import Counter, Histogram
import time
from typing import Callable
from fastapi import Request, Response

# Глобальные метрики (будут созданы один раз)
HTTP_REQUESTS_TOTAL = Counter(
    'http_requests_total',
    'Total HTTP requests',
    ['method', 'endpoint', 'status_code']
)
```

```
HTTP_REQUEST_DURATION = Histogram(
    'http_request_duration_seconds',
    'HTTP request duration in seconds',
    ['method', 'endpoint'],
    buckets=[0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
)

# Если нужны security-метрики
SECURITY_BLOCKED_REQUESTS = Counter(
    'security_requests_blocked_total',
    'Total blocked security requests',
    ['reason', 'source_ip']
)

SERVICE_ERRORS = Counter(
    'service_errors_total',
    'Total service errors',
    ['error_type', 'service']
)

def setup_metrics_middleware(app):
    """
    Настройка middleware для сбора метрик
    Использование: setup_metrics_middleware(app)
    """

    @app.middleware("http")
    async def metrics_middleware(request: Request, call_next:
Callable):
        # Начало измерения времени
        start_time = time.time()

        # Определяем endpoint (без query параметров)
        endpoint = request.url.path

        try:
            # Выполняем запрос
            response = await call_next(request)

            # Регистрируем успешный запрос
            HTTP_REQUESTS_TOTAL.labels(
                method=request.method,
                endpoint=endpoint,
                status_code=response.status_code
            ).inc()

            # Пример security логики (можно адаптировать)
            if endpoint.startswith('/admin') and not
request.headers.get('Authorization'):
                SECURITY_BLOCKED_REQUESTS.labels(
                    reason='unauthorized_admin_access',
                    source_ip=request.client.host if
request.client else 'unknown'
                ).inc()
```

```
        except Exception as e:
            # Регистрируем ошибку
            SERVICE_ERRORS.labels(
                error_type=type(e).__name__,
                service=app.title if hasattr(app, 'title') else
'unknown'
            ).inc()
            raise

        finally:
            # Всегда регистрируем длительность запроса
            duration = time.time() - start_time
            HTTP_REQUEST_DURATION.labels(
                method=request.method,
                endpoint=endpoint
            ).observe(duration)

        return response

    return app


# Экспорт метрик для /metrics endpoint
from prometheus_client import generate_latest, CONTENT_TYPE_LATEST
from fastapi.responses import Response

def metrics_endpoint():
    """
    Endpoint для получения метрик Prometheus
    Использование: @app.get("/metrics")
    """
    def endpoint():
        return Response(
            content=generate_latest(),
            media_type=CONTENT_TYPE_LATEST
        )
    return endpoint
```

10. dockerfile_template.txt

```
FROM python:3.9-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    g++ \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements
COPY requirements.txt .
```

```
# Install Python dependencies
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Create non-root user
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser

# Expose port
EXPOSE 80

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --
retries=3 \
    CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:80/health')"

# Run the application
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "80"]
```